HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

CƠ SỞ ĐÀO TẠO PHÍA BẮC

Khoa An toàn thông tin

BÁO CÁO BÀI TẬP LỚN

Môn học Cơ sở an toàn thông tin

MÔ PHỎNG TẤN CÔNG DDoS TẦNG ỨNG DỤNG VÀ TRIỂN KHAI GIẢI PHÁP PHÁT HIỆN GIẢM THIỂU BẰNG SURICATA IPS KẾT HỢP HAPROXY TRÊN PFSENSE

Giảng viên hướng dẫn: .....

Nhóm thực hiện: Nhóm 01

Lớp học phần: D24CQCE01-B

Học kỳ và năm học: Học kỳ 1, năm học 2026–2027

Hà Nội, tháng 11 năm 2026

DANH SÁCH THÀNH VIÊN VÀ PHÂN CÔNG

Báo cáo thuộc chủ đề "Kỹ thuật tấn công từ chối dịch vụ phân tán (DDoS) và giải pháp phát hiện, giảm thiểu tác động". Phần thực hành tập trung vào lưu lượng HTTP từ hai máy Kali, cơ chế chặn của Suricata IPS và phân phối yêu cầu bằng HAProxy trên pfSense.

Thành viên	Mã sinh viên	Công việc phụ trách	Sản phẩm và mức hoàn thành
Phạm Việt Toàn	B24DCCE269	Xây dựng lab VirtualBox gồm 2 Kali, pfSense và 2 web Ubuntu; cài đặt, cấu hình Suricata Inline IPS, HAProxy, firewall/NAT; viết kịch bản JMeter; chạy KB1–KB5; thu thập minh chứng; biên soạn báo cáo.	Hoàn thành 100%; báo cáo, script, ảnh minh chứng, JTL và log được lưu trong thư mục ddos-lab2.

Báo cáo do một thành viên thực hiện. Bảng phân công ghi nhận toàn bộ phần việc từ xây dựng môi trường, cấu hình công cụ, thực hiện phép thử, thu thập minh chứng đến biên soạn báo cáo.

TÓM TẮT

Tính sẵn sàng là một yêu cầu quan trọng của an toàn thông tin: dịch vụ phải tiếp tục phục vụ người dùng hợp lệ trong điều kiện vận hành đã xác định. Tấn công từ chối dịch vụ phân tán có thể làm suy giảm yêu cầu này bằng cách tạo lưu lượng từ nhiều nguồn và gây áp lực lên tài nguyên của hệ thống. Báo cáo nghiên cứu vấn đề trong phạm vi HTTP và xây dựng mô hình thực nghiệm có kiểm soát trên VirtualBox.

Mô hình gồm hai máy Kali Linux sử dụng Apache JMeter, một máy pfSense chạy đồng thời Suricata Inline IPS và HAProxy, cùng hai máy Ubuntu 24.04 cung cấp dịch vụ web bằng Python HTTP Server. Hai nguồn gửi yêu cầu tới cùng địa chỉ frontend. Suricata kiểm tra lưu lượng tại WAN của pfSense; HAProxy phân phối những yêu cầu được tiếp nhận tới các backend đang hoạt động. Cách bố trí này cho phép quan sát vai trò bổ sung của kiểm tra nội dung, chính sách theo ngưỡng và cân bằng tải.

Quá trình kiểm chứng được chia thành các phép thử độc lập: truy cập bình thường, đối chứng tắt/bật IPS đối với một URI kiểm tra, tạo lưu lượng đồng thời từ hai Kali khi chưa áp dụng chính sách ngưỡng, lặp lại với chính sách ngưỡng và kiểm tra khi một backend ngừng hoạt động. Dữ liệu được đối chiếu giữa JMeter, cảnh báo Suricata, HAProxy Stats và log HTTP của hai server.

Các phép thử đã hoàn tất trên môi trường thật. KB1 xác nhận cả hai nguồn nhận HTTP 200 qua frontend. KB2 cho thấy yêu cầu /ips-test.txt bị chặn khi IPS bật (timeout, 0 byte) và được phục vụ khi IPS tắt. KB3 ghi 80 mẫu, 0 lỗi; hai backend nhận 42 yêu cầu mỗi server. KB4 với rate_filter ghi 6 sự kiện DROP SID 1001002 và giảm lượng yêu cầu tải tới backend từ 80 xuống 72 (10%). KB5 ghi nhận web1 chuyển DOWN sau khoảng 3 giây, được đưa trở lại UP sau khoảng 4 giây, trong khi web2 phục vụ 4/4 yêu cầu thử. Báo cáo phân biệt kết quả đã quan sát với tiêu chí kiểm chứng và không suy rộng kết quả lab ra Internet.

Từ khóa: DDoS HTTP; Suricata; Inline IPS; HAProxy; pfSense; JMeter; cân bằng tải.

MỤC LỤC

MỞ ĐẦU **1**
1. Lý do lựa chọn đề tài 1
2. Mục tiêu thực hiện 1
3. Đối tượng và phạm vi nghiên cứu 1
4. Phương pháp thực hiện 1
5. Bố cục báo cáo 2
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ DDoS HTTP **3**
1.1. Khái niệm và đặc điểm của DDoS 3
1.2. Nguyên lý gây tải bằng yêu cầu HTTP 3
1.3. Tạo lưu lượng từ nhiều nguồn bằng JMeter 4
1.4. Các chỉ số sử dụng để đánh giá 4
CHƯƠNG 2. GIẢI PHÁP PHÁT HIỆN VÀ GIẢM THIỂU **6**
2.1. Vai trò của các thành phần 6
2.2. Cơ chế kiểm tra và chặn của Suricata 6
2.3. Cơ chế cân bằng tải của HAProxy 7
2.4. Phối hợp Suricata và HAProxy trên pfSense 7
2.5. Giới hạn và điều kiện đánh giá 8
CHƯƠNG 3. MÔI TRƯỜNG VÀ KIẾN TRÚC THỰC NGHIỆM **9**
3.1. Yêu cầu và thành phần môi trường 9
3.2. Kiến trúc mạng tổng thể 10
3.3. Quy hoạch IP và cổng dịch vụ 10
3.4. Luồng xử lý và phạm vi quan sát 11
3.5. Phối hợp hai nguồn tạo lưu lượng 11
3.6. Thu thập và đối chiếu dữ liệu 11
3.7. Điều kiện lặp lại và giới hạn đo 12
CHƯƠNG 4. CÀI ĐẶT VÀ CẤU HÌNH HỆ THỐNG **13**
4.1. Chuẩn bị năm máy ảo 13
4.2. Cấu hình mạng và địa chỉ IP 13
4.3. Triển khai dịch vụ trên hai web server Ubuntu 15
4.4. Cấu hình đường truy cập trên pfSense 16
4.5. Cài đặt và cấu hình HAProxy 17
4.6. Cài đặt và cấu hình Suricata Inline IPS 20
4.7. Cấu hình JMeter trên hai Kali 24
4.8. Xác nhận hệ thống trước thực nghiệm 25
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ **27**
5.1. Mục tiêu và quy trình thực nghiệm chung 27
5.2. Kịch bản 1 – Truy cập HTTP bình thường 27
5.3. Kịch bản 2 – Đối chứng khả năng chặn của IPS 29
5.4. Kịch bản 3 – Hai nguồn đồng thời khi chưa chặn theo ngưỡng 31
5.5. Kịch bản 4 – Hai nguồn đồng thời khi bật chính sách ngưỡng 34
5.6. Kịch bản 5 – Một backend ngừng hoạt động và khôi phục 36
5.7. Tổng hợp mức độ kiểm chứng 38
5.8. Đánh giá khả năng phối hợp Suricata và HAProxy 39
5.9. Hạn chế và yếu tố ảnh hưởng kết quả 39
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **41**
1. Kết luận 41
2. Hướng phát triển 41

TÀI LIỆU THAM KHẢO **42**
PHỤ LỤC A. HỒ SƠ CẤU HÌNH VÀ LẦN CHẠY **43**
A.1. Thông tin cần lưu cùng báo cáo 43
A.2. Quy tắc quản lý kết quả 43
PHỤ LỤC B. LỆNH KIỂM TRA VÀ THU LOG **44**
B.1. Trên Kali 44
B.2. Trên web server Ubuntu 44
PHỤ LỤC C. HỒ SƠ LUẬT SURICATA **45**
C.1. Phạm vi và trạng thái các luật 45
C.2. Phân biệt trạng thái đối chứng 45
PHỤ LỤC D. CẤU HÌNH HAPROXY TƯƠNG ĐƯƠNG **46**
PHỤ LỤC E. BIỂU MẪU TỔNG HỢP NHIỀU LẦN ĐO **47**

DANH MỤC HÌNH

Hình 3.1. Kiến trúc hai nguồn HTTP và hai backend qua pfSense....10
Hình 4.1. Danh sách năm máy ảo và tài nguyên thực cấp....13
Hình 4.2. Hai card mạng pfSense và ánh xạ Internal Network....14
Hình 4.3. Địa chỉ IP của hai Kali và hai web server....15
Hình 4.4. Hai Python HTTP Server và trang nhận diện backend....16
Hình 4.5. Alias KALI_LAB và luật WAN cho frontend....17
Hình 4.6. Backend pool và HTTP health check....18
Hình 4.7. Frontend HTTP tại WAN và backend mặc định....19
Hình 4.8. Trạng thái ban đầu của hai backend trên HAProxy Stats....20
Hình 4.9. Suricata được gắn vào WAN và chạy Inline IPS....21
Hình 4.10. Custom Rules áp dụng cho cả hai nguồn Kali....22
Hình 4.11. Chính sách ngưỡng và liên kết với instance WAN....23
Hình 4.12. Cấu hình JMeter tương ứng trên hai Kali....25
Hình 5.1. Phản hồi truy cập bình thường từ hai Kali....28
Hình 5.2. Log backend tương ứng và kết quả tải thấp....29
Hình 5.3. Yêu cầu nhận được nội dung khi IPS tắt....30
Hình 5.4. Kali không nhận nội dung khi luật chặn URI có hiệu lực....30
Hình 5.5. Suricata ghi nhận hành động chặn SID 1001001....31
Hình 5.6. Đối chiếu log backend và yêu cầu bình thường sau phép chặn....31
Hình 5.7. Hai Kali cùng chạy kế hoạch gửi tới frontend....32
Hình 5.8. Kết quả JMeter của KB3 trên từng nguồn....32
Hình 5.9. Yêu cầu thử nghiệm được nhận ở cả hai backend....33
Hình 5.10. Đối chiếu HAProxy Stats trước và sau KB3....33
Hình 5.11. Kết quả JMeter khi chính sách ngưỡng được bật....34
Hình 5.12. Bằng chứng chặn theo ngưỡng của cả hai IP nguồn....35
Hình 5.13. Lưu lượng tới backend và yêu cầu đối chứng khi chặn theo ngưỡng....35
Hình 5.14. Trạng thái hai backend trước khi tạo sự cố....37
Hình 5.15. Web 1 DOWN trong khi Web 2 tiếp tục phục vụ....37
Hình 5.16. Web 1 trở lại UP và tham gia nhận yêu cầu....37

DANH MỤC BẢNG

Bảng 1.1. Chỉ số và ý nghĩa trong thực nghiệm....4
Bảng 2.1. Đối chiếu vị trí kiểm tra và lựa chọn của đề tài....8
Bảng 3.1. Thông tin phần cứng và phiên bản thực nghiệm....9
Bảng 3.2. Kế hoạch địa chỉ của mô hình....10
Bảng 3.3. Nguồn dữ liệu và trường cần ghi nhận....11
Bảng 4.1. Bộ tham số kiểm chứng đề xuất cho hai Kali....24
Bảng 4.2. Điều kiện sẵn sàng và bằng chứng xác nhận....25
Bảng 5.1. Ma trận các kịch bản....27
Bảng 5.2. Kết quả của một lần chạy hai nguồn không áp dụng rate_filter....33
Bảng 5.3. So sánh hai trạng thái chính sách dưới cùng cấu hình tải....35
Bảng 5.4. Kết quả phép thử lỗi backend....38
Bảng 5.5. Phạm vi kết luận từ các minh chứng....38

DANH MỤC TỪ VIẾT TẮT

Thuật ngữ	Tên đầy đủ hoặc ý nghĩa
DDoS	Distributed Denial of Service – tấn công từ chối dịch vụ phân tán
HTTP	Hypertext Transfer Protocol – giao thức truyền tải siêu văn bản
IDS	Intrusion Detection System – hệ thống phát hiện xâm nhập
IPS	Intrusion Prevention System – hệ thống ngăn chặn xâm nhập
VM	Virtual Machine – máy ảo
WAN / LAN	Tên hai phía giao diện mạng của pfSense trong mô hình
NAT	Network Address Translation – chuyển đổi địa chỉ mạng
URI	Uniform Resource Identifier – định danh tài nguyên
SID	Signature ID – mã định danh luật Suricata
RPS	Requests per second – số yêu cầu trong một giây
JTL / JMX	Tệp kết quả thực nghiệm / tệp kế hoạch kiểm thử của JMeter
EVE JSON	Định dạng nhật ký sự kiện JSON của Suricata
Frontend	Điểm HAProxy lắng nghe yêu cầu từ phía người dùng
Backend	Máy chủ hoặc nhóm máy chủ thực sự xử lý yêu cầu
Health check	Phép kiểm tra định kỳ trạng thái backend

MỞ ĐẦU

1. Lý do lựa chọn đề tài

Một website có thể có dữ liệu chính xác và cơ chế xác thực tốt nhưng vẫn không đáp ứng mục đích sử dụng nếu người dùng không truy cập được. Khi nhiều nguồn đồng thời gửi yêu cầu, tài nguyên xử lý, số kết nối và các hàng đợi có thể trở thành giới hạn của dịch vụ. Vì vậy, bảo đảm tính sẵn sàng cần được xem xét cùng với việc phát hiện lưu lượng bất thường và tổ chức năng lực phục vụ.

Nhóm lựa chọn DDoS HTTP vì loại lưu lượng này gắn trực tiếp với một dịch vụ web có thể xây dựng trong phòng thực hành. Các yêu cầu có thể đúng cú pháp nhưng được gửi với cường độ không phù hợp. Đây là tình huống thuận lợi để khảo sát mối quan hệ giữa phát hiện theo ngưỡng, ngăn chặn và phân phối tải, thay vì chỉ chứng minh một công cụ có thể phát sinh nhiều gói tin. Cách tiếp cận phòng vệ nhiều lớp cũng phù hợp với định hướng giảm thiểu tác động đối với tính sẵn sàng được CERT-EU trình bày [1].

2. Mục tiêu thực hiện

Mục tiêu của đề tài là xây dựng và kiểm chứng một chuỗi xử lý HTTP gồm hai nguồn tạo lưu lượng, một điểm kiểm tra IPS và một nhóm hai backend. Các câu hỏi thực nghiệm là: yêu cầu có đi qua đúng frontend hay không; Suricata có thực sự chặn lưu lượng khớp chính sách hay chỉ ghi cảnh báo; HAProxy có phân phối lưu lượng đến cả hai server; và dịch vụ có tiếp tục đáp ứng khi một backend ngừng hoạt động hay không.

Để trả lời các câu hỏi trên, báo cáo xác định rõ điều kiện đầu vào, cấu hình giữ nguyên giữa các lần chạy và loại bằng chứng cần thu thập. Kết quả giảm thiểu chỉ được đánh giá trong phạm vi môi trường, tải và luật đã kiểm tra; không suy rộng thành khả năng chống DDoS trên Internet.

3. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là lưu lượng HTTP rõ trên cổng 8080, cơ chế kiểm tra HTTP của Suricata, chính sách chuyển hành động theo ngưỡng và cân bằng tải ở tầng ứng dụng bằng HAProxy. Hai Kali có địa chỉ 10.10.10.10 và 10.10.10.11 cùng gửi yêu cầu tới 10.10.10.1:8080. Các backend tại 10.10.20.20 và 10.10.20.21 phục vụ cùng tập tài nguyên thử nghiệm.

JMeter đóng vai trò bộ tạo tải có kiểm soát. Mô hình không triển khai phần mềm botnet hoặc cơ chế chiếm quyền thiết bị. Việc có hai địa chỉ nguồn thể hiện tính phân tán ở mức phòng thực hành; mức suy giảm dịch vụ phải được chứng minh riêng bằng phép đo. Đề tài không triển khai phản xạ, khuếch đại, khai thác giao thức khác hay phân tích HTTP bên trong TLS.

4. Phương pháp thực hiện

Nhóm sử dụng phương pháp thực nghiệm có đối chứng. Trước hết, từng thành phần được kiểm tra ở mức chức năng. Sau đó, cùng một kế hoạch gửi yêu cầu được chạy trong hai trạng thái chính sách: chưa chuyển hành động sang chặn và đã bật chuyển hành động theo ngưỡng. Bằng chứng thu được từ nhiều vị trí giúp phân biệt lỗi máy chủ, lỗi mạng, sự chặn của IPS và tác dụng của cân bằng tải.

Ảnh chụp được gắn với mã lần chạy và nội dung cần chứng minh. Bên dưới mỗi ảnh có phần giải thích, nhận xét và đánh giá. Số liệu trong báo cáo được lấy từ JTL, log backend và sự kiện Suricata của đúng mã lần chạy; ảnh, JTL và log gốc được lưu trong thư mục ddos-lab2 của nhóm.

5. Bố cục báo cáo

Chương 1 trình bày cơ sở lý thuyết trực tiếp phục vụ thực nghiệm DDoS HTTP. Chương 2 giải thích giải pháp phối hợp Suricata và HAProxy. Chương 3 mô tả kiến trúc, địa chỉ mạng và cách đo. Chương 4 trình bày quá trình cài đặt, cấu hình và xác nhận hệ thống. Chương 5 tổ chức các kịch bản, đối chiếu minh chứng và đánh giá kết quả. Phần cuối gồm kết luận, tài liệu tham khảo và các phụ lục phục vụ tái lập.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VỀ DDoS HTTP

1.1. Khái niệm và đặc điểm của DDoS

DDoS là hành vi phối hợp nhiều nguồn nhằm làm cho tài nguyên hoặc dịch vụ không còn đáp ứng được nhu cầu của người sử dụng hợp lệ [1]. Trong phạm vi HTTP, tác động có thể biểu hiện ở thời gian phản hồi tăng, kết nối bị từ chối, yêu cầu chờ quá lâu hoặc phản hồi lỗi. Tài nguyên bị gây áp lực có thể nằm ở web server, proxy, thiết bị kiểm tra lưu lượng hoặc hạ tầng mạng trước máy chủ.

Hai đặc điểm cần được xem xét riêng là sự phân tán của nguồn gửi và tác động tới tính sẵn sàng. Trong mô hình này, sự phân tán được xác định bằng hai IP nguồn độc lập cùng hướng tới một frontend trong khoảng thời gian chồng lấn. Tác động tới dịch vụ được xác định bằng số liệu phản hồi và khả năng phục vụ, không chỉ bằng số máy tham gia.

Một lần gửi 40 yêu cầu có thể đủ để kiểm chứng phân phối tải hoặc kích hoạt một ngưỡng thấp được đặt riêng cho bài thực hành. Tuy nhiên, số lượng này không tự chứng minh website bị quá tải. Vì vậy, thuật ngữ "mô phỏng DDoS HTTP" trong báo cáo mô tả cách tổ chức nguồn và lưu lượng thử nghiệm; kết luận về gián đoạn chỉ được đưa ra khi có dữ liệu tương ứng.

1.2. Nguyên lý gây tải bằng yêu cầu HTTP

1.2.1. Quá trình xử lý và tài nguyên liên quan

Đối với mỗi yêu cầu, phía phục vụ phải tiếp nhận kết nối, đọc thông tin HTTP, xác định tài nguyên và gửi phản hồi. Trong mô hình reverse proxy, quá trình còn có bước HAProxy chọn một backend và chuyển yêu cầu đến backend đó. Nếu tốc độ yêu cầu tới lớn hơn năng lực xử lý trong một khoảng thời gian, hàng đợi có thể tăng và người dùng phải chờ lâu hơn.

Với Python HTTP Server phục vụ file tĩnh, công việc phía ứng dụng tương đối đơn giản. Mô hình phù hợp để quan sát đường đi và kết quả HTTP, nhưng không đại diện cho mọi ứng dụng có cơ sở dữ liệu, xác thực hay xử lý nghiệp vụ. Nếu file đã nằm trong bộ nhớ đệm hệ điều hành, mức tải CPU và đĩa cũng khác với một website động.

1.2.2. Tốc độ gửi và mức đồng thời

Số yêu cầu, tốc độ yêu cầu và số tác vụ đồng thời là ba đại lượng khác nhau. Cùng 80 yêu cầu, việc gửi trải đều trong một phút tạo áp lực khác với việc dồn trong vài giây. Nhiều luồng JMeter có thể làm tăng số yêu cầu đang chờ, nhưng tốc độ thực tế còn phụ thuộc vào độ trễ, timer, giới hạn tài nguyên và timeout.

Với mô hình gửi và chờ phản hồi thông thường, mỗi luồng thường phải hoàn tất một mẫu trước khi chuyển sang mẫu tiếp theo. Khi IPS khiến một số yêu cầu chờ hết timeout, nguồn tạo tải có thể tự gửi chậm lại. Bởi vậy, throughput giảm sau khi bật IPS cần được giải thích bằng cả số yêu cầu được backend nhận lẫn cơ chế chờ ở phía JMeter.

1.2.3. Phân biệt với truy cập hợp lệ

Một yêu cầu GET hợp lệ không mang sẵn thuộc tính "tấn công". Cùng một đường dẫn có thể được truy cập bởi người dùng thật, công cụ giám sát hoặc nguồn tạo tải. Ngưỡng được đặt quá thấp có thể ảnh hưởng cả truy cập hợp lệ; ngưỡng đặt quá cao có thể không phát hiện được tổng lưu lượng từ nhiều nguồn.

Trong thực nghiệm, nhóm chủ động giới hạn địa chỉ nguồn và URI kiểm tra để tạo điều kiện đo rõ ràng. Đây là chính sách dành cho lab. Khả năng khái quát sang hệ thống thật còn phụ thuộc vào việc đo hành vi truy cập bình thường, đặc điểm tài nguyên và chính sách chấp nhận rủi ro.

1.3. Tạo lưu lượng từ nhiều nguồn bằng JMeter

JMeter sử dụng Thread Group để quy định số luồng người dùng ảo, thời gian tăng số luồng và số lần lặp. HTTP Request mô tả yêu cầu gửi tới dịch vụ. Timer bổ sung thời gian chờ và Listener thu thập kết quả. Các thành phần này cho phép xây dựng một kế hoạch có thể lưu thành tệp JMX và thực hiện lặp lại [2].

Trong mô hình, hai máy Kali giữ cùng địa chỉ đích, đường dẫn và các tham số tải; mỗi máy có nhãn nguồn riêng. Hai kế hoạch được khởi chạy gần cùng thời điểm và kiểm tra bằng dấu thời gian thực tế. Đây là hai tiến trình tạo tải độc lập. Chỉ sử dụng tên gọi JMeter Remote Testing khi thực sự triển khai bộ điều khiển và các máy chạy từ xa theo cơ chế của JMeter.

Nếu một Thread Group gồm T luồng, L vòng lặp và đúng một HTTP Request ở mỗi vòng, số mẫu dự kiến trên một máy là $N = T \times L$. Công thức này cần điều chỉnh nếu có nhiều sampler, vòng lặp bổ sung, dừng sớm hoặc cấu hình chạy theo thời gian. Tổng hai nguồn bằng $N1 + N2$; số thực tế được xác nhận bằng tệp kết quả của từng nguồn.

GUI phù hợp để xây dựng và kiểm tra kế hoạch nhỏ. Khi thu số liệu tải lặp lại, chế độ dòng lệnh giúp giảm chi phí giao diện; các Listener lưu quá nhiều nội dung phản hồi cũng cần được hạn chế [3]. Trong mọi trường hợp, báo cáo phải ghi đúng chế độ đã sử dụng, bởi bản thân máy phát tải có thể là điểm nghẽn.

1.4. Các chỉ số sử dụng để đánh giá

Bảng 1.1. Chỉ số và ý nghĩa trong thực nghiệm

Chỉ số	Cách xác định	Ý nghĩa và giới hạn
Tổng mẫu N	Số mẫu hoàn tất trong JTL của từng Kali	Phân biệt với số dự kiến theo Thread Group
Tỷ lệ lỗi	Mẫu không thành công / tổng mẫu × 100%	Có thể do timeout, IPS, backend hoặc cấu hình
Thời gian đáp ứng	Thời gian hoàn tất từng mẫu; xét trung bình và P95	Cần tách nhóm thành công và nhóm timeout
Throughput	Số mẫu hoàn tất chia khoảng đo	Phụ thuộc thời gian chờ của nguồn tạo tải

Chỉ số	Cách xác định	Ý nghĩa và giới hạn
Yêu cầu tới backend	Log HTTP lọc đúng URI và mã lần chạy	Loại health check và truy cập ngoài lần chạy
Phân bố backend	Nweb1 và Nweb2 trong cùng lần chạy	Kiểm chứng hai server cùng tham gia xử lý
Hành động chặn	SID, thời gian, nguồn, đích và action trong log IPS	Cảnh báo đơn thuần chưa đủ chứng minh chặn
Trạng thái dịch vụ	Phản hồi của yêu cầu đối chứng và health check	UP chỉ thể hiện điều kiện health check đạt

Thời gian đáp ứng trung bình được tính bằng tổng thời gian hoàn tất chia số mẫu. P95 là mức thời gian mà khoảng 95% mẫu không vượt quá. Nếu cần gộp kết quả hai Kali, phải tính từ tập mẫu gốc; không lấy trung bình cộng hai giá trị P95. Tương tự, tỷ lệ lỗi chung bằng tổng số mẫu lỗi chia tổng số mẫu của cả hai nguồn.

Gọi Nweb1 và Nweb2 là số yêu cầu thử nghiệm được ghi tại mỗi backend. Tỷ lệ phân phối của web1 là Nweb1/(Nweb1 + Nweb2) × 100%, nếu tổng khác 0. Độ lệch có thể mô tả bằng |Nweb1 − Nweb2|/(Nweb1 + Nweb2) × 100%. Đây là phép đo phân phối, chưa phải phép đo tăng năng lực xử lý.

Khi so sánh hai trạng thái chính sách với số yêu cầu đầu vào thực tế tương đương, mức giảm yêu cầu tới backend có thể được tính từ chênh lệch tổng log của hai lần chạy. Không gọi đại lượng này là “tỷ lệ IPS chặn chính xác”, bởi mất kết nối, thử lại và giới hạn thời gian đều có thể ảnh hưởng đến kết quả.

CHƯƠNG 2. GIẢI PHÁP PHÁT HIỆN VÀ GIẢM THIỂU

2.1. Vai trò của các thành phần

pfSense là điểm kết nối giữa hai mạng nội bộ và là nơi triển khai dịch vụ kiểm tra, phân phối lưu lượng. Các luật firewall xác định đường truy cập được phép; Suricata thực hiện kiểm tra HTTP và chính sách chặn; HAProxy tiếp nhận yêu cầu tại frontend và chọn backend. Việc phân định vai trò giúp tránh quy kết mọi yêu cầu thất bại cho cùng một công cụ.

HAProxy tổ chức điểm lắng nghe phía khách và nhóm server phía sau, phù hợp với việc đưa một dịch vụ web ra một địa chỉ truy cập chung [7]. Trong mô hình này, tường lửa của web server không được sử dụng làm cơ chế phát hiện DDoS. Nếu hệ điều hành cần quy tắc cho phép TCP/8080 từ pfSense, đó là điều kiện kết nối tới dịch vụ, không phải kết quả phòng chống của Suricata.

2.2. Cơ chế kiểm tra và chặn của Suricata

2.2.1. Inline IPS và vị trí trên đường truyền

Suricata ở chế độ Inline IPS tham gia vào đường đi của lưu lượng để có thể thực thi hành động chặn. Trên pfSense, chế độ này sử dụng netmap và phụ thuộc khả năng hỗ trợ của driver giao diện. Vì vậy, đánh dấu "Enable" hoặc nhìn thấy tiến trình chạy chưa đủ: cần có luật thích hợp và một phép thử xác nhận gói hoặc luồng thực sự bị chặn. Mã nguồn giao diện cấu hình của gói Suricata cũng phân biệt luật ALERT với luật DROP trong Inline Mode [11].

Trong thiết kế, Suricata kiểm tra lưu lượng đi vào WAN trước khi HAProxy xử lý yêu cầu HTTP ở frontend. Cụm từ "lọc trước" diễn tả thứ tự kiểm tra ở đường vào, không có nghĩa HAProxy hoàn toàn không tiêu tốn tài nguyên cho một kết nối bị chặn sau đó. Chẳng hạn, bắt tay TCP có thể đã hoàn tất trước khi nội dung URI đủ để khớp luật.

2.2.2. Luật, SID và hành động

Một luật được xác định bởi phần điều kiện mạng và phần điều kiện nội dung. Địa chỉ nguồn giúp khoanh vùng hai Kali; địa chỉ đích là WAN của pfSense; cổng 8080 xác định dịch vụ; các bộ đệm HTTP cho phép kiểm tra phương thức hoặc URI. Suricata hỗ trợ các từ khóa như `http.method` và `http.uri` để kiểm tra đúng trường của giao thức [5].

SID giúp truy ngược một sự kiện về luật sinh ra nó. Trong đề tài, SID 1001001 dành cho phép kiểm tra chặn URI; SID 1001002 dành cho lưu lượng kiểm tra theo ngưỡng. Tách hai SID giúp xác định rõ nguyên nhân của mỗi hành vi. `alert` phục vụ ghi nhận; `drop` là hành động chặn khi được thực thi trong chế độ IPS phù hợp. Việc chỉ thấy tên luật ở trang Alerts không thay thế việc xem hành động.

2.2.3. Kiểm chứng chặn URI

Luật kiểm tra URI được thiết kế khớp đường dẫn `/ips-test.txt`. Khi luật có hiệu lực, một yêu cầu duy nhất cũng có thể bị chặn. Phép thử này trả lời câu hỏi Suricata có nằm đúng trên đường truyền và có thực thi DROP hay không. Nó không cần làm quá tải máy chủ và không phải là bằng chứng phát hiện DDoS theo cường độ.

Các chuỗi như `IPS_ON_001` hoặc `IPS_OFF_001` được thêm vào query string để đối chiếu log giữa các máy. Chuỗi này là nhãn thực nghiệm. Nếu luật khớp phần đầu `/ips-test.txt`, việc đổi nhãn không làm yêu cầu thoát khỏi luật. Trạng thái tắt/bật trong phép đối chứng phải được thay đổi ở IPS hoặc luật, không chỉ thay chữ trong URL.

2.2.4. Chính sách chuyển hành động theo ngưỡng

Chính sách thực nghiệm sử dụng một luật ALERT nhận diện GET tới `/load-test.txt`, kết hợp `rate_filter`. Khi số lần luật khớp đạt điều kiện cấu hình trong khoảng theo dõi, hành động được chuyển sang DROP trong thời gian timeout quy định. `track by_src` theo dõi riêng theo IP nguồn; nó không tự cộng tổng hai IP thành một bộ đếm chung [6].

Ngưỡng 20 lần khớp trong 10 giây và thời gian chuyển hành động 15 giây được chọn làm tham số kiểm chứng chức năng trong lab. Các giá trị này không được suy ra từ năng lực thực tế của Python server. Một chính sách vận hành thật cần có số liệu nền và quá trình hiệu chỉnh để cân bằng giữa phát hiện và chặn nhầm.

Vì theo dõi riêng từng nguồn, cần kiểm tra kết quả cho cả 10.10.10.10 và 10.10.10.11. Nếu mỗi IP gửi dưới ngưỡng nhưng tổng lưu lượng vượt năng lực backend, chính sách hiện tại có thể không kích hoạt. Đây là giới hạn trực tiếp của thiết kế, đồng thời là lý do không kết luận đã giải quyết đầy đủ vấn đề DDoS phân tán.

2.3. Cơ chế cân bằng tải của HAProxy

Frontend cung cấp điểm truy cập duy nhất `10.10.10.1:8080`. Backend pool gồm web1 và web2 với cùng trọng số. Thuật toán round-robin phân phối lượt chọn theo vòng giữa các server đủ điều kiện [8]. Tỷ lệ quan sát có thể không tuyệt đối 50/50 do điều kiện kết nối, tái sử dụng kết nối, thay đổi trạng thái hoặc truy cập khác trong thời gian đo. Bởi vậy, log có mã lần chạy đáng tin cậy hơn việc nhìn riêng một bộ đếm tích lũy.

Health check là lưu lượng do HAProxy tạo để kiểm tra backend. Đề tài dùng GET `/health.txt` và kiểm tra HTTP 200. Khi số lần kiểm tra thất bại liên tiếp đạt điều kiện cấu hình, backend bị loại khỏi danh sách phục vụ; khi đạt số lần thành công cần thiết, server được đưa trở lại [9]. Các yêu cầu health check phải được loại khỏi số yêu cầu do JMeter tạo.

HAProxy còn giúp chuyển yêu cầu mới sang backend còn hoạt động khi một server gặp sự cố. Điều này không bảo đảm các yêu cầu đang xử lý trên server hỏng được bảo toàn hoặc người dùng hoàn toàn không thấy lỗi trong thời gian phát hiện. Bằng chứng cần có chuỗi trạng thái UP, DOWN, UP cùng phản hồi HTTP trong các giai đoạn tương ứng.

2.4. Phối hợp Suricata và HAProxy trên pfSense

Thiết kế lựa chọn một pfSense có hai card mạng. Luồng từ phía nguồn đi vào WAN, được Suricata kiểm tra, sau đó những yêu cầu được tiếp nhận bởi frontend HAProxy mới được chuyển tới LAN và một backend. Với reverse proxy thông thường, server thấy địa chỉ kết nối từ pfSense LAN là 10.10.20.1. Vì vậy, việc Python log không hiển thị trực tiếp IP Kali không có nghĩa lưu lượng đã mất nguồn ở phía WAN.

Bảng 2.1. Đối chiếu vị trí kiểm tra và lựa chọn của đề tài

Phương án	Ý nghĩa	Đánh giá đối với mô hình
IPS ở đường vào trước HAProxy	Quan sát nguồn Kali trước kết nối proxy tới backend	Phù hợp kiểm tra theo IP nguồn và URI frontend
Kiểm tra chủ yếu ở phía backend	Quan sát yêu cầu sau khi proxy đã xử lý	HAProxy vẫn tiếp nhận tải; nguồn kết nối thường là proxy
IPS và HAProxy cùng pfSense	Kết hợp hai vai trò tại một VM, IPS gắn WAN	Ít VM bổ sung, dễ đối chiếu log; chia sẻ CPU và RAM

Ba cách mô tả trên không hoàn toàn loại trừ nhau: "cùng pfSense" nói về nơi triển khai, còn "trước HAProxy" nói về vị trí kiểm tra. Đề tài kết hợp hai đặc điểm này. Cấu hình cần bảo đảm yêu cầu tới cổng 8080 không bị một NAT Port Forward cũ chuyển thẳng đến backend. Port forward có thể thay đổi đích trước khi dịch vụ cục bộ của firewall xử lý [10].

Firewall WAN chỉ cho phép hai Kali truy cập đúng frontend; không cần mở một đường cho Kali tới trực tiếp backend. Web GUI quản trị được truy cập từ LAN. Nhờ vậy, việc nhìn thấy log ở backend và frontend có thể được giải thích theo cùng một luồng đã thiết kế.

2.5. Giới hạn và điều kiện đánh giá

Suricata và HAProxy bổ sung cho nhau: thành phần thứ nhất quyết định đối với lưu lượng khớp chính sách, thành phần thứ hai sử dụng nhóm backend để phục vụ yêu cầu được tiếp nhận. Cân bằng tải không tự xác định mọi yêu cầu có mục đích gây hại; lọc theo URI hoặc ngưỡng cũng không tạo thêm băng thông cho đường truyền.

Một pfSense duy nhất là điểm tập trung tài nguyên và điểm sự cố chung. Nếu CPU, RAM hoặc đường vào của nó bị bão hòa, việc có hai backend không khắc phục được toàn bộ vấn đề. Tương tự, khi các VM cùng chạy trên một máy thật, sự cạnh tranh tài nguyên có thể làm máy phát tải và máy phục vụ cùng chậm. Những yếu tố này phải được ghi nhận để tránh diễn giải sai kết quả.

Do thử nghiệm sử dụng HTTP rõ, Suricata có thể kiểm tra URI ở WAN. Khi thay bằng HTTPS, nội dung ứng dụng được mã hóa và mô hình cần thiết kế lại vị trí giải mã, kiểm tra hoặc giới hạn lưu lượng. Báo cáo không suy rộng kết quả kiểm tra URI HTTP sang HTTPS.

CHƯƠNG 3. MÔI TRƯỜNG VÀ KIẾN TRÚC THỰC NGHIỆM

3.1. Yêu cầu và thành phần môi trường

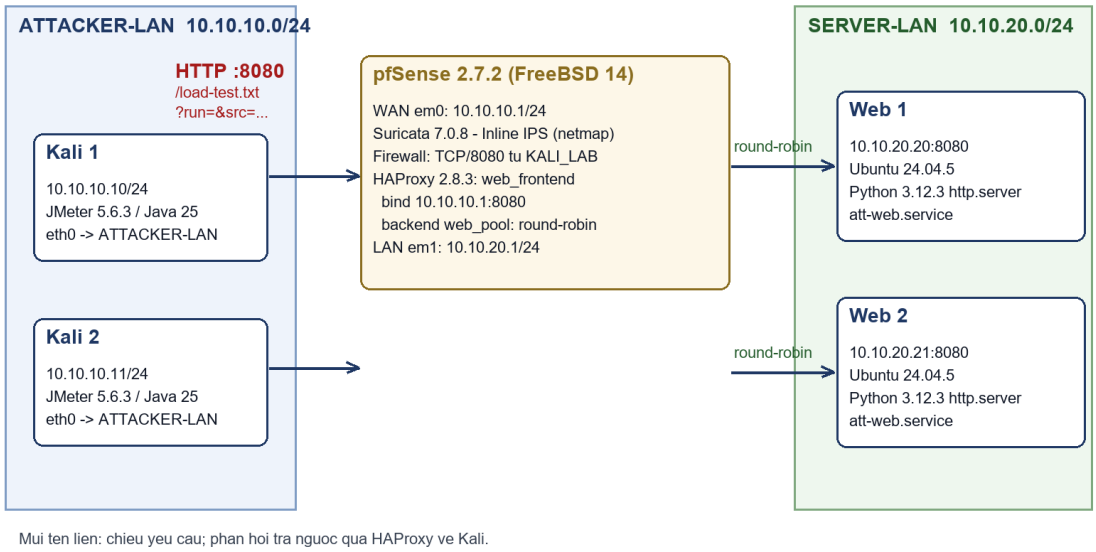
Môi trường gồm năm máy ảo: hai Kali, một pfSense và hai web server Ubuntu 24.04. Các VM được nối qua hai Internal Network độc lập. Chỉ pfSense có giao diện ở cả hai mạng. Trong giai đoạn đo, lưu lượng được giới hạn trong lab; các card NAT phục vụ tải phần mềm được ngắt hoặc tắt trước khi chạy kịch bản.

Môi trường phải đủ tài nguyên để các VM hoạt động ổn định đồng thời. Gói kiểm tra lưu lượng có nhu cầu bộ nhớ riêng ngoài hệ điều hành và các dịch vụ khác [4]. Không dùng các mức RAM hoặc CPU dự kiến như số liệu đã đo. Bảng 3.1 được điền từ VirtualBox Settings và thông tin của máy thật trước khi thực nghiệm; thay đổi tài nguyên giữa các lần đối chứng phải được ghi rõ.

Bảng 3.1. Thông tin phần cứng và phiên bản thực nghiệm

Thành phần	CPU và RAM thực cấp	Hệ điều hành hoặc phiên bản
Máy thật	Core i7-1185G7 4 nhân/8 luồng; 15,7 GB RAM; ổ C: còn trống 21,5 GB	Windows 11 Pro 10.0.26200; VirtualBox 7.2.14
Kali 1	2 vCPU; 2048 MB	Kali GNU/Linux Rolling; Java 25.0.3; JMeter 5.6.3
Kali 2	2 vCPU; 2048 MB	Kali GNU/Linux Rolling; Java 25.0.3; JMeter 5.6.3
pfSense	2 vCPU; 1536 MB	pfSense 2.7.2-RELEASE; Suricata 7.0.8; HAProxy 2.8.3
Web 1	1 vCPU; 768 MB	Ubuntu 24.04.5 LTS; Python 3.12.3; systemd att-web.service
Web 2	1 vCPU; 768 MB	Ubuntu 24.04.5 LTS; Python 3.12.3; systemd att-web.service

3.2. Kiến trúc mạng tổng thể



Hình 3.1. Kiến trúc hai nguồn HTTP và hai backend qua pfSense

Giải thích: Hai Kali cùng truy cập một dịch vụ tại WAN của pfSense. Suricata và HAProxy nằm trên cùng máy ảo nhưng đảm nhiệm hai bước khác nhau. Các backend chỉ nối với mạng SERVER-LAN. Mũi tên mô tả chiều xử lý yêu cầu; phản hồi được trả lại qua HAProxy tới phía nguồn.

Nhận xét và đánh giá: Cách tách hai mạng giúp kiểm soát đường đi và giảm khả năng gửi nhầm trực tiếp đến backend. Sơ đồ thể hiện một dịch vụ có hai backend, không phải hai cuộc thử nghiệm độc lập mỗi Kali chỉ hướng đến một server. Mô hình phù hợp để quan sát cơ chế, song vẫn phụ thuộc tài nguyên của pfSense và máy thật.

3.3. Quy hoạch IP và cổng dịch vụ

Bảng 3.2. Kế hoạch địa chỉ của mô hình

Máy hoặc giao diện	Mạng VirtualBox	Địa chỉ IPv4	Gateway hoặc vai trò
Kali 1	ATTACKER-LAN	10.10.10.10/24	10.10.10.1
Kali 2	ATTACKER-LAN	10.10.10.11/24	10.10.10.1
pfSense WAN	ATTACKER-LAN	10.10.10.1/24	Không đặt upstream gateway trong lab cô lập
pfSense LAN	SERVER-LAN	10.10.20.1/24	Điểm quản trị và phía backend
Web 1	SERVER-LAN	10.10.20.20/24	10.10.20.1
Web 2	SERVER-LAN	10.10.20.21/24	10.10.20.1

Trong mô hình này, tên WAN chỉ giao diện phía Kali, không đồng nghĩa với một địa chỉ Internet công cộng. Hai máy cùng mạng phải dùng IP riêng; sao chép VM mà giữ nguyên địa chỉ sẽ gây xung đột. Ký hiệu /24 tương ứng subnet mask 255.255.255.0. Địa chỉ loopback 127.0.0.1 luôn chỉ chính máy đang sử dụng nó, vì vậy Kali không thể dùng loopback của mình để mở server trên máy web.

Frontend và hai HTTP server đều dùng cổng 8080, nhưng trên các địa chỉ khác nhau. Web GUI pfSense được quản trị qua `https://10.10.20.1`. Các phép thử dùng IP trực tiếp nên không phụ thuộc phân giải DNS trong mạng cô lập.

3.4. Luồng xử lý và phạm vi quan sát

Một yêu cầu từ Kali có địa chỉ nguồn .10 hoặc .11, đích 10.10.10.1:8080. Suricata tại WAN quan sát nguồn thật này và kiểm tra điều kiện HTTP. Nếu yêu cầu không bị chặn, HAProxy tiếp nhận và chọn web1 hoặc web2. Kết nối từ HAProxy tới Python server đi qua phía LAN, do đó log ứng dụng thường ghi 10.10.20.1 làm địa chỉ khách.

Các yêu cầu /health.txt phát sinh từ HAProxy tới backend phục vụ giám sát trạng thái. Chúng không chứng minh Kali đã gửi được yêu cầu và không được tính vào tổng mẫu JMeter. Để đối chiếu một lần chạy, URI cần có nhãn như run=KB3_R2&src=K1; khi đó log ứng dụng vẫn phân biệt được nguồn thử nghiệm dù địa chỉ TCP tại backend là pfSense.

3.5. Phối hợp hai nguồn tạo lưu lượng

Hai Kali sử dụng cùng bản test plan với các tham số kiểm tra tương đương. Điểm khác biệt là nhãn `src=K1` hoặc `src=K2` và tên tệp kết quả. Nhóm thống nhất mã lần chạy, kiểm tra giờ hệ thống, sau đó khởi chạy trên cả hai máy. Dấu thời gian trong JTL được dùng để xác nhận khoảng giao nhau thực tế, thay vì chỉ căn cứ việc bấm nút gần nhau.

Với phép kiểm chứng ban đầu, mỗi nguồn dự kiến tạo 40 yêu cầu bằng hai luồng và 20 vòng lặp, mỗi vòng một HTTP Request. Tổng dự kiến là 80 yêu cầu. Đây là bộ tham số nhỏ để kiểm tra chính sách lab; mọi lần dừng sớm hoặc mẫu bổ sung phải được ghi trong bảng kết quả. Giá trị 40 của các ảnh thử nghiệm cũ không tự động trở thành 40 cho mỗi máy của mô hình mới.

Khoảng đồng thời được tính từ thời điểm bắt đầu muộn hơn giữa hai nguồn tới thời điểm kết thúc sớm hơn. Nếu khoảng này không tồn tại, lần chạy chưa đáp ứng điều kiện hai nguồn hoạt động đồng thời. Việc hai VM cùng bật nguồn không đủ để thay thế điều kiện này.

3.6. Thu thập và đối chiếu dữ liệu

Bảng 3.3. Nguồn dữ liệu và trường cần ghi nhận

Nguồn	Nội dung cần lưu	Mục đích đối chiếu
JMeter K1 và K2	JMX; JTL; giờ bắt đầu và kết thúc; số mẫu; lỗi; thời gian đáp ứng	Xác nhận đầu vào và kết quả tại từng nguồn

Nguồn	Nội dung cần lưu	Mục đích đối chiếu
Suricata	SID, action, nguồn, đích, cổng, thời gian; EVE nếu có	Xác định luật nào đã khớp hoặc chặn
HAProxy Stats	Trạng thái, LastChk, bộ đếm trước/sau, thời gian chụp	Đánh giá backend tham gia xử lý và lỗi
Python HTTP Server	Log của cả web1, web2 chứa URI và mã lần chạy	Xác định yêu cầu đã tới ứng dụng
Máy thật và pfSense	CPU, RAM, sự cố treo hoặc khởi động lại	Nhận biết ảnh hưởng tài nguyên ngoài chính sách

Nhật ký EVE có thể chứa nhiều loại sự kiện khác nhau; khi phân tích cần chọn đúng sự kiện cảnh báo và trường hành động thay vì chỉ đếm mọi dòng JSON [14]. Một cảnh báo có thể liên quan đến một luồng, trong khi một luồng có nhiều gói; do đó không chuyển trực tiếp số cảnh báo thành số yêu cầu bị chặn.

Ảnh chụp được đặt tên theo cấu trúc `KBx_Ry_thietbi_noidung.png`. Cần giữ bản gốc và tạo bản đánh dấu để đưa vào báo cáo. Màu xanh dùng cho phản hồi thành công hoặc UP; màu đỏ cho DROP, DOWN hoặc lỗi cần phân tích. Mỗi ảnh có thêm khoanh vùng, nhãn hoặc mũi tên để thông tin vẫn hiểu được khi in đen trắng.

3.7. Điều kiện lặp lại và giới hạn đo

Giữa hai trạng thái so sánh, giữ nguyên số backend, trọng số, nội dung file, timeout, số luồng, số vòng lặp, timer và tài nguyên VM. KB3 và KB4 được chạy lặp hai lần; lần R2 có đủ JTL, log và ảnh Stats nên được dùng làm kết quả chính, R1 dùng để đối chiếu độ ổn định.

Trước mỗi lần chạy, dừng tải cũ, kiểm tra backend UP và làm sạch phần hiển thị kết quả JMeter. Không xóa tệp JTL đang cần làm minh chứng. Khi thay đổi chính sách, cần chờ trạng thái chặn cũ hết hiệu lực và xác nhận các kết nối cũ không làm sai lệch phép đo. Chỉ xóa các state của luồng thử nghiệm khi cần, tránh làm gián đoạn phiên quản trị.

Đồng hồ giữa các VM có sai lệch: trong các lần chạy, pfSense chậm hơn Kali khoảng 2–4 phút. Báo cáo dùng giờ UTC trong JTL để xác định khoảng chồng lấn và dùng giờ pfSense để mô tả sự kiện Suricata, sau khi đã ghi nhận độ lệch. Các kết quả thu trong thời gian Kali bị treo, host thiếu RAM hoặc pfSense khởi động lại được đánh dấu riêng và không trộn vào lần đo ổn định.

CHƯƠNG 4. CÀI ĐẶT VÀ CẤU HÌNH HỆ THỐNG

4.1. Chuẩn bị năm máy ảo

Hai máy Kali được sử dụng làm nguồn tạo lưu lượng. Khi tạo Kali 2 từ bản clone của Kali 1, chọn tạo lại địa chỉ MAC cho card mạng và đổi IPv4 sang 10.10.10.11 trước khi đưa vào cùng mạng. Hai web server Ubuntu cũng phải có MAC và IP riêng. Việc clone giúp giữ phần mềm tương đương, nhưng không thay thế việc kiểm tra cấu hình trên từng bản sao.

pfSense được tạo thành một VM riêng với hai card mạng. Sau khi khởi động từ bộ cài, cài hệ điều hành lên đĩa ảo, gỡ bộ cài khỏi ổ quang ảo và khởi động lại. Tại console, đối chiếu MAC của card với VirtualBox để gán đúng WAN và LAN; không suy đoán giao diện chỉ từ thứ tự tên em0, em1.

Trong giai đoạn cài đặt gói, có thể tạm nối WAN pfSense vào NAT để lấy mạng; khi đó WAN nhận cấu hình phù hợp với mạng NAT. Sau khi tải đủ Suricata, HAProxy và phần mềm trên các VM, tắt VM, chuyển kết nối về hai Internal Network của mô hình và đặt IP tĩnh theo Bảng 3.2. Chỉ bắt đầu đo khi WAN đã trở về 10.10.10.1/24, không còn phụ thuộc gateway NAT cũ.

VBoxManage list vms / cau hinh tai nguyen thuc cap

```
$ VBoxManage list vms | grep ATT
  "ATT-Kali1"    "ATT-Kali2"    "ATT-pfSense"    "ATT-WEB1"    "ATT-WEB2"

May ao        vCPU   RAM      Card 1 (Internal)   Card 2
ATT-Kali1     2      2048 MB  ATTACKER-LAN        NAT 10.0.3.15
ATT-Kali2     2      2048 MB  ATTACKER-LAN        NAT 10.0.3.15
ATT-pfSense   2      1536 MB  ATTACKER-LAN        SERVER-LAN
ATT-WEB1      1       768 MB  SERVER-LAN          NAT 10.0.3.15
ATT-WEB2      1       768 MB  SERVER-LAN          NAT 10.0.3.15

# May that: Lenovo 20XW003KUS; Intel Core i7-1185G7 (4C/8T); 15.7 GB RAM;
# Windows 11 Pro 10.0.26200; VirtualBox 7.2.14
```

Hình 4.1. Danh sách năm máy ảo và tài nguyên thực cấp

Giải thích: Hình ghi nhận các thành phần thực sự có trong môi trường và tài nguyên được cấp cho chúng. **Nhận xét:** Cần có đủ năm VM theo kiến trúc; trạng thái Running chỉ xác nhận VM đang chạy, chưa xác nhận dịch vụ bên trong sẵn sàng. **Đánh giá:** Các phép đo chỉ so sánh được khi tài nguyên không thay đổi giữa các kịch bản hoặc mọi thay đổi được ghi lại.

4.2. Cấu hình mạng và địa chỉ IP

4.2.1. Kết nối VirtualBox

Tắt hoàn toàn VM trước khi thay đổi card mạng. Trong Settings → Network, đặt card của hai Kali vào Internal Network tên ATTACKER-LAN. Đặt card của hai web server vào SERVER-LAN. Với pfSense, card WAN nối ATTACKER-LAN và card LAN nối SERVER-LAN. Bật Enable Network Adapter và Cable Connected ở các card sử dụng. Hai tên mạng phải trùng chính xác giữa những VM cần nối cùng nhau.

Nếu Adapter 2 không sửa được, kiểm tra VM đã Powered Off hay vẫn ở trạng thái Running hoặc Saved. Có thể đổi giao diện Settings sang chế độ hiển thị đầy đủ nếu phiên bản VirtualBox đang ẩn

các mục nâng cao. Không cần đặt Kali hoặc Windows vào cả hai mạng, vì điều đó có thể tạo đường truy cập ngoài kiến trúc đã chọn.

Internal Network cua pfSense va cac may dau cuoi

```
ATT-pfSense  Adapter 1 (WAN, em0): Internal Network "ATTACKER-LAN"  MAC 080027BE145F
             Adapter 2 (LAN, em1): Internal Network "SERVER-LAN"    MAC 080027B7BF1D
ATT-Kali1    Adapter 1: "ATTACKER-LAN"   MAC 0800275A87BC
ATT-Kali2    Adapter 1: "ATTACKER-LAN"   MAC 0800279CD558
ATT-WEB1     Adapter 1: "SERVER-LAN"     MAC 080027FA8787
ATT-WEB2     Adapter 1: "SERVER-LAN"     MAC 08002705DA6D

# Ten Internal Network phai trung tuyet doi; card NAT chi dung de cap nhat goi.
```

Hình 4.2. Hai card mạng pfSense và ánh xạ Internal Network

Giải thích: Hai card của pfSense tạo liên kết giữa hai phân đoạn, trong khi các máy đầu cuối chỉ gắn với phân đoạn của mình. **Nhận xét:** Cùng subnet IP nhưng khác tên Internal Network vẫn có thể không liên lạc được. **Đánh giá:** Ảnh này chứng minh kết nối ảo đã được thiết kế; cần kết hợp kiểm tra IP và HTTP ở các bước sau để xác nhận hoạt động.

4.2.2. Cấu hình pfSense và các máy đầu cuối

Tại pfSense console, gán WAN và LAN theo MAC rồi đặt IP tĩnh. WAN dùng 10.10.10.1/24, LAN dùng 10.10.20.1/24. Trong lab này không đặt upstream gateway cho WAN. Từ web server ở SERVER-LAN, mở https://10.10.20.1 để quản trị. Dùng IP tĩnh ở các máy đầu cuối giúp cấu hình không phụ thuộc DHCP.

Trên Kali, sử dụng Network Manager để chọn IPv4 Manual; điền IP .10 hoặc .11, prefix 24 và gateway 10.10.10.1. Trên web server Ubuntu, cấu hình IPv4 tĩnh bằng netplan với địa chỉ .20 hoặc .21, prefix 24 và gateway 10.10.20.1. Sau khi áp dụng, xác nhận bằng các lệnh sau trên đúng hệ điều hành.

```
# Terminal trên từng Kali
ip -br addr
ip route

# Trên từng web server Ubuntu
ip -br addr; ip route
```

Tại Interfaces → WAN của pfSense, kiểm tra mục Block private networks. Vì WAN lab nhận lưu lượng có nguồn 10.10.10.0/24, cần bỏ chặn mạng riêng trên riêng giao diện WAN của mô hình này. Đây là điều chỉnh phù hợp với mạng thực hành có địa chỉ private; không phải quy tắc chung cho mọi WAN Internet.

```
ip -br addr / ip route tren bon may dau cuoi

KALI 1 (10.10.10.10)
eth0             UP             10.10.10.10/24
KALI 2 (10.10.10.11)
eth0             UP             10.10.10.11/24 fe80::a00:27ff:fe9c:d558/64
route: 10.10.10.0/24 dev eth0 proto kernel scope link src 10.10.10.11

WEB 1 (10.10.20.20) - Ubuntu 24.04.5
eth0             UP             10.10.20.20/24
WEB 2 (10.10.20.21) - Ubuntu 24.04.5
eth0             UP             10.10.20.21/24

# Gateway cua Kali: 10.10.10.1; gateway cua web: 10.10.20.1 (pfSense).
```

Hình 4.3. Địa chỉ IP của hai Kali và hai web server

Giải thích: Hình xác nhận hai nguồn .10 và .11, hai backend .20 và .21 thuộc đúng mạng. Nhận xét: Hai web server chạy Ubuntu 24.04 với địa chỉ tĩnh, không dùng DHCP nên không xuất hiện địa chỉ 169.254.x.x. Đánh giá: Hai IP nguồn độc lập là điều kiện cần của mô phỏng phân tán, nhưng chưa chứng minh hai nguồn đã gửi đồng thời.

4.2.3. Kiểm tra kết nối cơ bản

Từ từng web server, mở Web GUI pfSense tại LAN. Sau khi triển khai HTTP server, dùng Diagnostics → Test Port trên pfSense để kiểm tra 10.10.20.20:8080 và 10.10.20.21:8080. Từ Kali, dùng HTTP qua frontend sau khi HAProxy đã được cấu hình. Không dùng riêng kết quả ping thất bại để kết luận mạng sai, vì ICMP có thể chưa được firewall cho phép.

4.3. Triển khai dịch vụ trên hai web server Ubuntu

Hai web server chạy Ubuntu 24.04.5 LTS với Python 3.12.3. Mỗi máy được cloud-init tạo sẵn thư mục /opt/www và dịch vụ systemd att-web.service. Web 1 phục vụ nội dung nhận diện SERVER 1, Web 2 là SERVER 2; hai máy cung cấp cùng bốn tệp kiểm tra.

```
sudo mkdir -p /opt/www
printf 'WEB SERVER 1\n' | sudo tee /opt/www/index.html
printf 'OK SERVER 1\n' | sudo tee /opt/www/health.txt
printf 'IPS TEST - SERVER 1\n' | sudo tee /opt/www/ips-test.txt
printf 'LOAD TEST - SERVER 1\n' | sudo tee /opt/www/load-test.txt
ExecStart=/usr/bin/python3 -m http.server 8080
```

Trên Web 2, thay các nhãn SERVER 1 bằng SERVER 2. Dịch vụ chạy với WorkingDirectory=/opt/www, lắng nghe cổng 8080 trên mọi địa chỉ và được systemd tự khởi động lại khi gặp lỗi. Log được đọc từ journal của att-web.service nên vẫn thu được các dòng GET sau khi dịch vụ khởi động lại.

Kiểm tra cục bộ trên mỗi web server bằng http://127.0.0.1:8080/health.txt, sau đó kiểm tra từ pfSense. Nếu cục bộ hoạt động nhưng pfSense không kết nối được, kiểm tra IP, card mạng và luật cho phép TCP/8080 từ 10.10.20.1. Không tắt toàn bộ tường lửa chỉ để thay thế việc xác định nguyên nhân kết nối. Python HTTP Server phù hợp cho lab phục vụ file đơn giản và không được thiết kế như máy chủ triển khai sản xuất [12].

```
Python HTTP Server tren hai backend (phan hoi tu pfSense)

$ curl -s http://10.10.20.20:8080/     # tu pfSense LAN
WEB SERVER 1
SERVER 1
Host: web1 (10.10.20.20)

$ curl -s http://10.10.20.21:8080/
WEB SERVER 2
SERVER 2
Host: web2 (10.10.20.21)

# Noi dung thuc te: &lt;html&gt;&lt;head&gt;&lt;title&gt;ATT DDoS Lab&lt;/title&
# Noi dung thuc te: &lt;html&gt;&lt;head&gt;&lt;title&gt;ATT DDoS Lab&lt;/title&
```

Hình 4.4. Hai Python HTTP Server và trang nhận diện backend

Giải thích: Hai dịch vụ hoạt động độc lập trên hai IP, có khả năng trả file kiểm tra. **Nhận xét:** Phản hồi cục bộ chỉ chứng minh ứng dụng đang chạy; phản hồi từ pfSense mới xác nhận đường LAN tới backend. **Đánh giá:** Phải hoàn tất kiểm tra này trước khi phân tích lỗi HAProxy hoặc Suricata để tránh quy kết sai lỗi ứng dụng cho IPS.

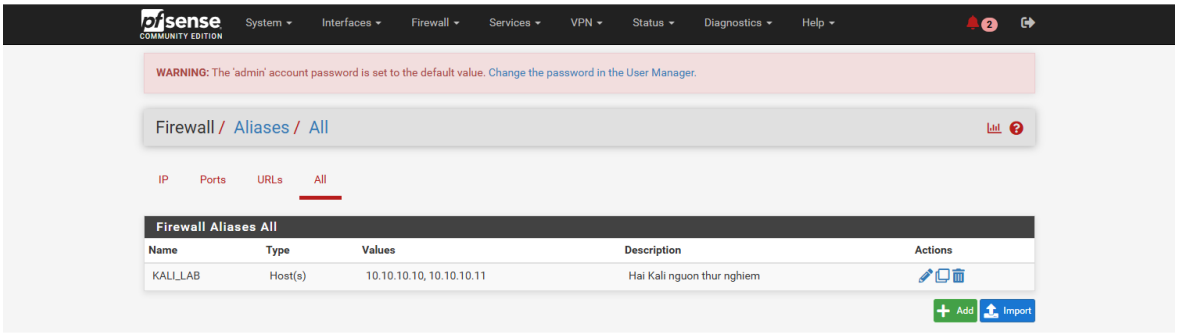
4.4. Cấu hình đường truy cập trên pfSense

Tạo alias `KALI_LAB` tại Firewall → Aliases, chứa đúng hai host 10.10.10.10 và 10.10.10.11. Tại Firewall → Rules → WAN, thêm luật Pass với giao thức TCP, nguồn KALI_LAB, đích WAN address và cổng đích 8080. Lưu và Apply Changes. Không cần dùng luật WAN allow any to any để phục vụ kịch bản này.

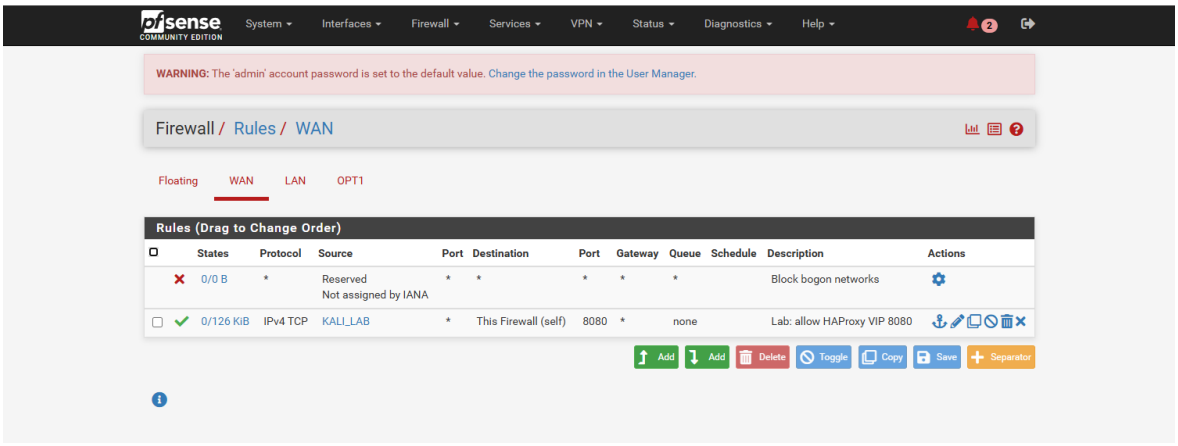
Nếu đã có Port Forward từ WAN:8080 tới một web server, vào Firewall → NAT → Port Forward, mở luật tương ứng, bật Disable rồi lưu và áp dụng. Kiểm tra cả firewall rule liên quan do NAT tạo ra. Port forward cũ có thể làm yêu cầu đi tới một backend thay vì frontend HAProxy [10]. Giữ nguyên các luật LAN đang phục vụ quản trị.

Vô hiệu hóa các luật WAN cũ cho phép Kali truy cập trực tiếp 10.10.20.20 hoặc 10.10.20.21 nếu chúng không thuộc thiết kế mới. Nếu sau thay đổi vẫn còn kết nối theo đường cũ, vào Diagnostics → States, lọc đúng IP và cổng của phép thử rồi xóa riêng các state tương ứng. Không xóa toàn bộ bảng state như một bước bắt buộc của mọi lần chạy.

(a) Alias KALI_LAB: host 10.10.10.10 va 10.10.10.11



(b) Rule WAN TCP/8080: source KALI_LAB -> WAN address



Hình 4.5. Alias KALI_LAB và luật WAN cho frontend

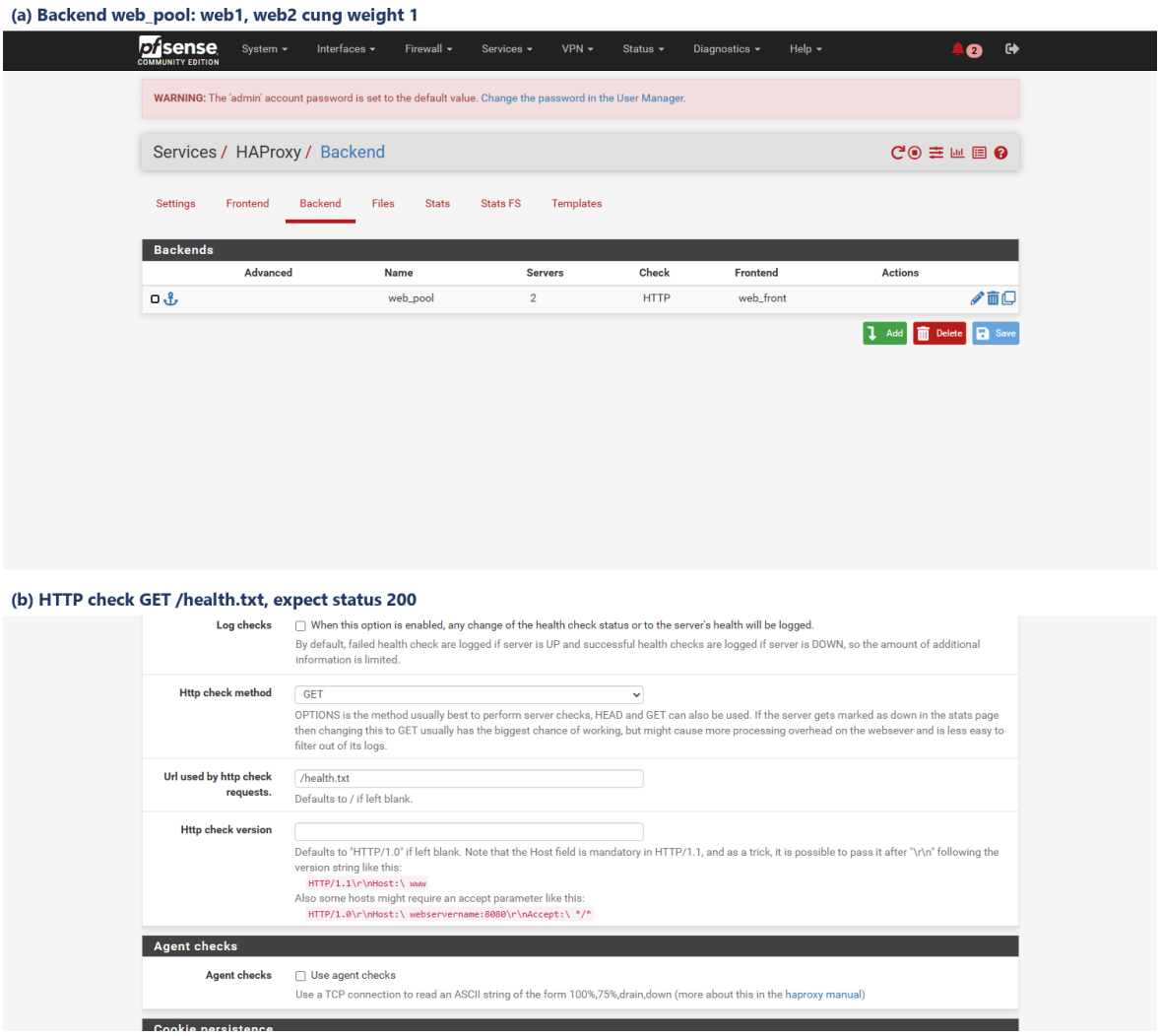
Giải thích: Firewall giới hạn các nguồn thử nghiệm vào điểm truy cập chung. **Nhận xét:** Việc thêm Kali 2 phải được phản ánh trong alias hoặc rule; một luật chỉ chứa .10 không cho phép suy ra .11 cũng đã được kiểm tra. **Đánh giá:** Cấu hình này hỗ trợ quy kết yêu cầu backend cho đường qua HAProxy; cần kết hợp log thực tế để xác nhận.

4.5. Cài đặt và cấu hình HAProxy

4.5.1. Backend pool và kiểm tra sức khỏe

Tại System → Package Manager → Available Packages, cài gói HAProxy phù hợp với phiên bản pfSense. Sau đó mở Services → HAProxy, bật dịch vụ trong Settings và cấu hình backend. Trong giao diện gói, tạo backend tên `web_pool`, chế độ HTTP, thuật toán round-robin. Thêm web1 tại 10.10.20.20:8080 và web2 tại 10.10.20.21:8080, cùng trọng số 1; không đặt một server làm backup trong phép thử phân phối đều.

Thiết lập HTTP health check với phương thức GET, đường dẫn `/health.txt` và điều kiện phản hồi 200. Một cấu hình kiểm chứng có thể dùng khoảng kiểm tra 2 giây, fall 3, rise 2; báo cáo phải điền đúng giá trị đang được áp dụng nếu khác. Tên trường và đơn vị trong Web GUI cần được đối chiếu trước khi lưu, tránh nhầm 2 giây với 2 mili giây. Cơ chế fall/rise quyết định số lần thất bại hoặc thành công liên tiếp trước khi đổi trạng thái [9].



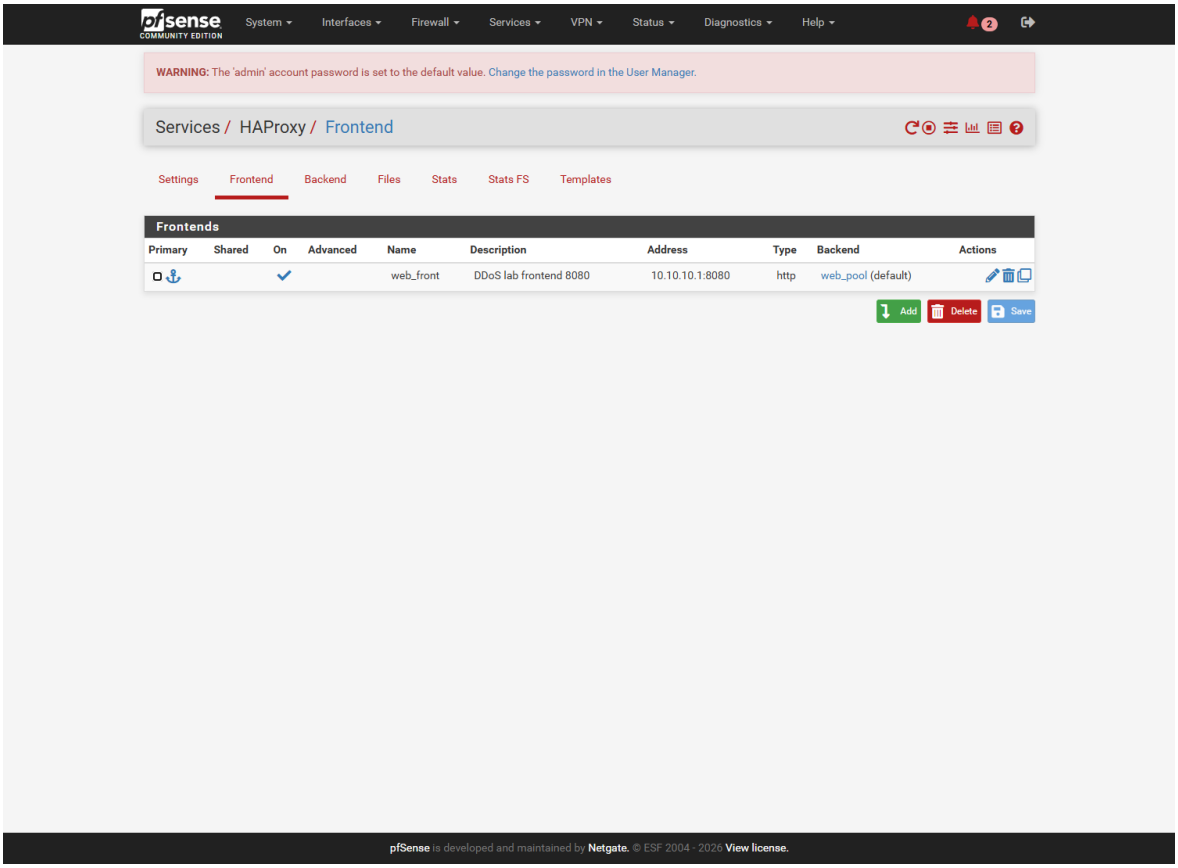
Hình 4.6. Backend pool và HTTP health check

Giải thích: Backend pool xác định các đích mà HAProxy có thể chọn. Health check tách server không đáp ứng ra khỏi vòng phân phối. **Nhận xét:** Cả hai phải là server hoạt động, không phải một active và một backup, nếu mục tiêu là cùng nhận tải. **Đánh giá:** Đây là bằng chứng cấu hình; khả năng cân bằng và chuyển đổi cần được xác nhận bằng các kịch bản ở Chương 5.

4.5.2. Frontend và liên kết tới backend

Tạo frontend tên web_front, địa chỉ lắng nghe WAN address 10.10.10.1, cổng 8080, loại HTTP. Chọn web_pool làm backend mặc định. Lưu, Apply và xác nhận dịch vụ HAProxy chạy không báo lỗi. Không cấu hình chuyển hướng tới backend bằng NAT để thay cho bước gán backend này.

Kiểm tra URL `http://10.10.10.1:8080/` từ Kali. Khi các yêu cầu được phân phối, nội dung SERVER 1 hoặc SERVER 2 cho biết backend trả lời. Với một số ít yêu cầu, chưa cần đòi hỏi thứ tự luân phiên tuyệt đối; cần kiểm tra cả log và trạng thái server.



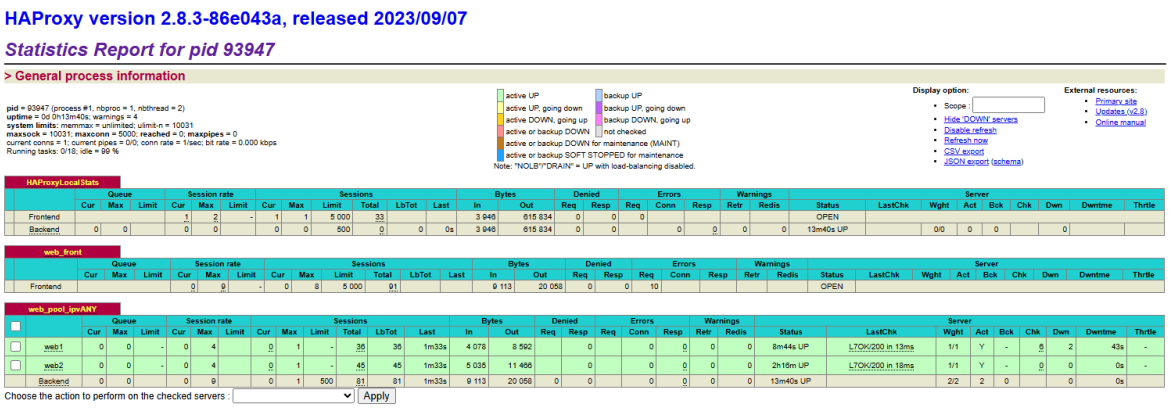
Hình 4.7. Frontend HTTP tại WAN và backend mặc định

Giải thích: Frontend xác định đích duy nhất của cả hai Kali. Nhận xét: Gửi JMeter thẳng tới IP backend sẽ bỏ qua mục tiêu kiểm chứng HAProxy của báo cáo. Đánh giá: Cấu hình khớp với kiến trúc khi đích nguồn là WAN:8080 và backend pool chứa hai web server.

4.5.3. Cách đọc trang Stats

Trong Services → HAProxy → Stats, quan sát nhóm backend web_pool hoặc tên được gói sinh ra tương ứng. Tập trung vào hai dòng web1 và web2: Status phải là UP; LastChk phản ánh kiểm tra HTTP thành công, thường hiển thị L7OK/200. Xem các cột lỗi và hàng đợi để phát hiện bất thường. Các bộ đếm Total là tích lũy; muốn phân tích một lần chạy phải chụp trước và sau.

`Sessions Total` là bộ đếm phiên, không mặc nhiên bằng số HTTP Request khi kết nối được tái sử dụng. Nếu Stats không cung cấp bộ đếm yêu cầu HTTP phù hợp ở mức từng server, sử dụng log `/load-test.txt` có mã lần chạy để xác định số yêu cầu. Bộ đếm trước/sau giúp hỗ trợ đối chiếu, không thay thế log ứng dụng.



Hình 4.8. Trạng thái ban đầu của hai backend trên HAProxy Stats

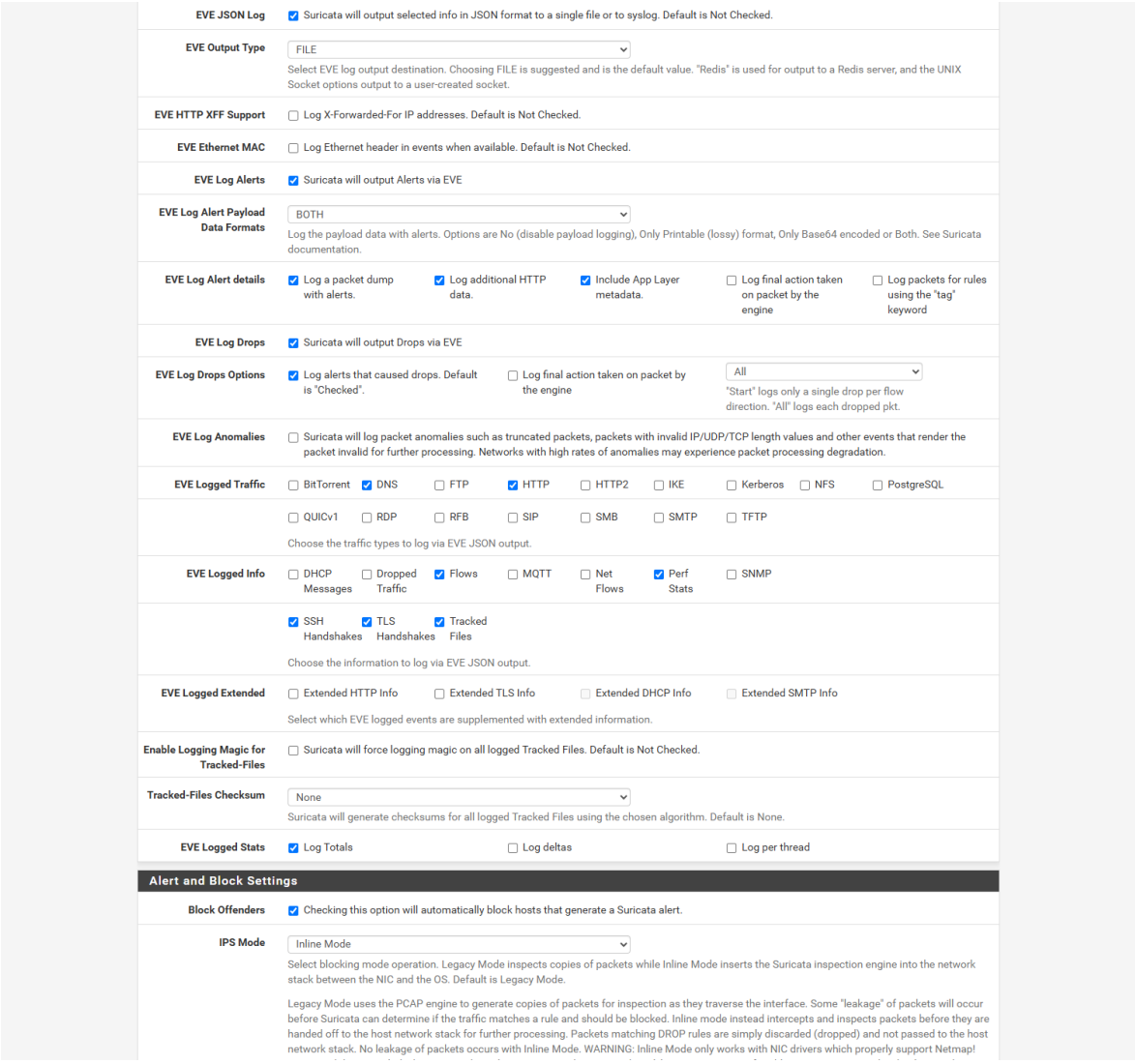
Giải thích: Hình xác định hệ thống ở trạng thái sẵn sàng trước phép đo. **Nhận xét:** UP chỉ thể hiện health check đạt tại thời điểm quan sát. **Đánh giá:** Nếu một backend DOWN ngay từ đầu, phép thử sau đó không còn là phép cân bằng trên hai server và phải xử lý nguyên nhân trước.

4.6. Cài đặt và cấu hình Suricata Inline IPS

4.6.1. Bật giao diện kiểm tra

Cài Suricata tại Package Manager, mở Services → Suricata → Interfaces và thêm WAN. Bật giao diện, chọn Inline IPS Mode ở phần thiết lập IPS và thực hiện các tùy chọn bật chặn mà phiên bản gói yêu cầu. Kiểm tra log khởi động để xác nhận chế độ inline hoạt động và giao diện hỗ trợ netmap [11]. Chỉ việc có tiến trình Suricata đang chạy chưa xác định được nó chạy ở chế độ nào.

Với card ảo được driver em hỗ trợ, cần kiểm tra tương thích thực tế của phiên bản pfSense và VirtualBox. Nếu có lỗi offloading hoặc netmap, đối chiếu các tùy chọn hardware checksum offloading, TSO và LRO tại System → Advanced → Networking; ghi lại thiết lập đã áp dụng. Không coi promiscuous mode là điều kiện mặc định bắt buộc cho topology định tuyến này. Những thay đổi mạng phải được kiểm chứng lại bằng truy cập bình thường trước khi đo IPS.



Hình 4.9. Suricata được gắn vào WAN và chạy Inline IPS

Giải thích: Giao diện WAN là nơi Suricata có thể nhìn thấy hai IP nguồn trước reverse proxy. **Nhận xét:** Chế độ IDS hoặc chỉ ghi alert chưa đáp ứng mục tiêu ngăn chặn. **Đánh giá:** Cần bổ sung kết quả DROP thực tế ở kịch bản 2 để xác nhận cấu hình có hiệu lực.

4.6.2. Khai báo mạng được bảo vệ

Trong phần Lists của Suricata, tạo danh sách mạng dùng cho HOME_NET, gồm `10.10.10.1/32` và `10.10.20.0/24`. `/32` chỉ đúng một địa chỉ WAN của frontend; `/24` chỉ mạng backend. Khi tạo danh sách thủ công, bỏ các lựa chọn tự thêm mạng, gateway, DNS, VPN hoặc VIP không thuộc phạm vi dự kiến. Gán danh sách này vào Home Net của Suricata WAN.

HOME_NET là biến mô tả mạng được bảo vệ, không phải danh sách tự động bỏ qua mọi kiểm tra. Không đưa hai Kali vào Pass List hay tạo luật pass rộng làm vô hiệu phép thử. Các luật lab dưới đây dùng địa chỉ nguồn và đích tường minh để kết quả dễ đối chiếu với kiến trúc.

4.6.3. Hai luật thử nghiệm

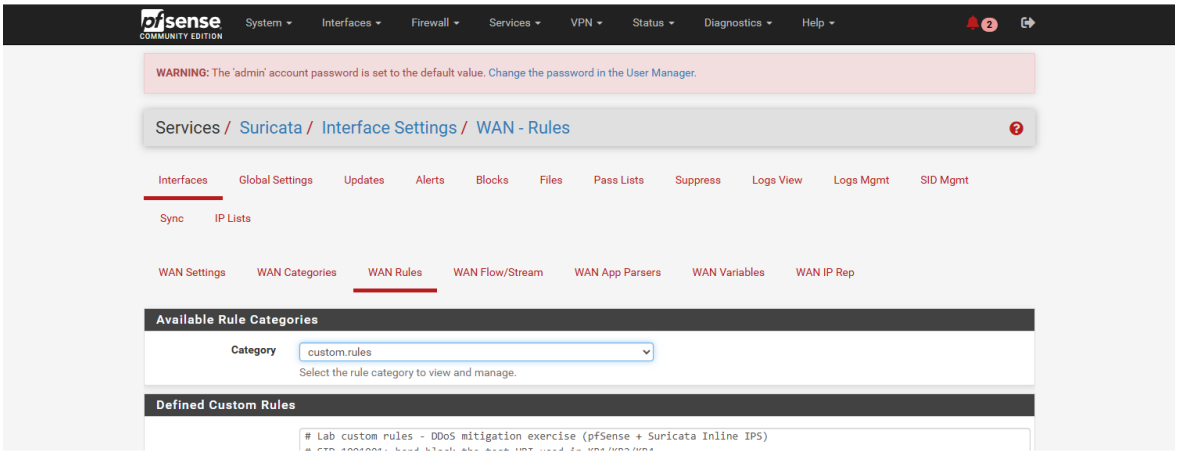
Tại Rules của giao diện WAN, mở nhóm Custom Rules hoặc trình soạn thảo luật tùy chỉnh tương ứng. Sao lưu các luật cũ. Những luật lab cũ trùng mục tiêu hoặc có thể chặn trước phép thử được tạm vô hiệu hóa; nếu chỉnh dạng văn bản, thêm # ở đầu toàn bộ dòng luật cần comment. Không comment hoặc xóa các cấu hình không liên quan.

Thêm hai luật dưới đây, mỗi luật là một dòng logic. Trong Word, dòng dài có thể được hiển thị xuống hàng; khi nhập vào trình soạn thảo, giữ nguyên nội dung và không chèn ký tự ngắt lệnh ngoài cú pháp.

```
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB BLOCK GET /ips-test.txt from external";
flow:to_server,established; content:"/ips-test.txt"; http_uri; nocase; classtype:policy-
violation; sid:1001001; rev:1;)
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB ALERT /load-test.txt high rate";
flow:to_server,established; content:"/load-test.txt"; http_uri; nocase; classtype:policy-
violation; sid:1001002; rev:1;)
```

Các luật dùng $EXTERNAL_NET và $HOME_NET; phạm vi hai nguồn Kali được giới hạn bằng alias KALI_LAB trong luật firewall WAN. Luật thứ nhất chặn đường dẫn kiểm tra; luật thứ hai chỉ ghi nhận trước khi có chính sách chuyển hành động. Hai SID này không trùng với luật khác trong cấu hình của nhóm.

(a) WAN Rules - custom.rules sau khi luu



(b) Noi dung day du hai luat 1001001 va 1001002

custom.rules - hai luat cua bai thi nghiem

```
# Lab custom rules - DDoS mitigation exercise (pfSense + Suricata Inline IPS)
# SID 1001001: hard-block the test URI used in KB1/KB2/KB4
drop http $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB BLOCK GET /ips-test.txt from external"; flow:to_server,established; content:"/ips-
    test.txt"; http_uri; nocase; classtype:policy-violation; sid:1001001; rev:1;)
# SID 1001002: alert on the load-test URI; rate_filter turns it into a drop (see suppress list)
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"LAB ALERT /load-test.txt high rate"; flow:to_server,established; content:"/load-test.txt";
    http_uri; nocase; classtype:policy-violation; sid:1001002; rev:1;)
```

Hình 4.10. Custom Rules áp dụng cho cả hai nguồn Kali

Giải thích: SID 1001001 và 1001002 phục vụ hai câu hỏi thực nghiệm khác nhau. **Nhận xét:** DROP của luật URI không chứng minh chính sách theo ngưỡng đã kích hoạt. **Đánh giá:** Việc tách luật giúp tránh dùng một ảnh chặn tĩnh để kết luận phát hiện DDoS theo tốc độ.

4.6.4. Chính sách rate_filter

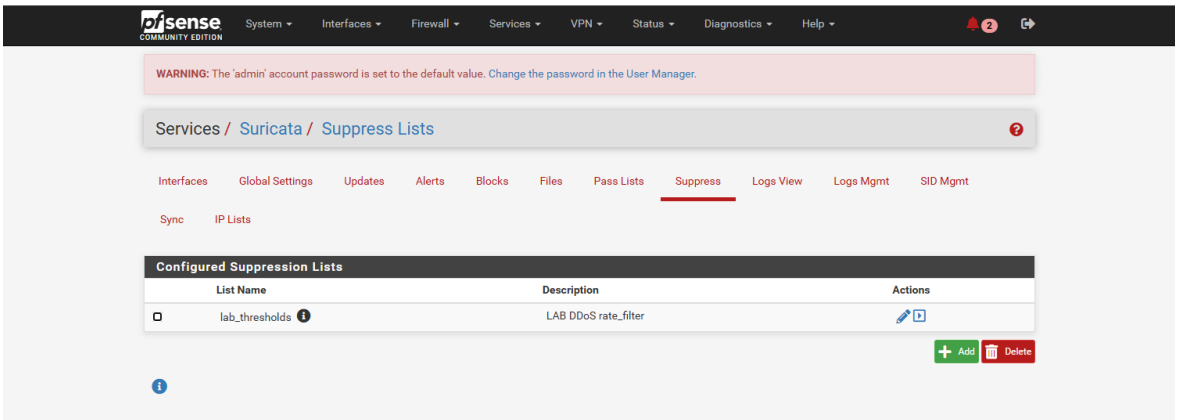
Ở phần Suppress của gói Suricata, tạo một danh sách dành cho lab, ví dụ `LAB_RATE_POLICY`, và nhập cấu hình threshold bổ sung bên dưới. Trong gói pfSense, nội dung này được chuyển vào cấu hình lọc/ngưỡng; cần gán đúng danh sách cho WAN tại mục Alert Suppression and Filtering. Tên tab hoặc trường có thể khác theo phiên bản, vì vậy phải xác nhận danh sách được instance WAN nạp và không có lỗi cú pháp.

```
rate_filter gen_id 1, sig_id 1001002, track by_src, count 20, seconds 10, new_action drop,
timeout 15
```

Trong cú pháp này, SID liên kết tới luật HTTP thứ hai; bộ đếm theo từng địa chỉ nguồn; `new_action drop` xác định hành động trong thời gian chính sách có hiệu lực [6]. Dòng cấu hình không phải luật chữ ký thông thường và không được dán vào vị trí Custom Rules như một luật `alert http`. Cũng không thay nó bằng `suppress`, vì suppress dùng để điều khiển cảnh báo chứ không đồng nghĩa với chính sách chặn theo ngưỡng.

Sau khi lưu và áp dụng, khởi động lại instance WAN khi phiên bản gói yêu cầu. Kiểm tra log khởi động và nội dung cấu hình hiệu lực. Mã nguồn gói cho thấy danh sách bổ sung được lưu dưới dạng nội dung cấu hình; khả năng nhận dòng rate_filter phải được xác nhận ở đúng bản đang cài [15]. Nếu có lỗi, xử lý lỗi cấu hình trước khi chạy tải, không kết luận IPS chặn thành công từ một kết nối vốn đã hỏng.

(a) Suppression list lab_thresholds da gan cho WAN



(b) Noi dung rate_filter trong threshold.config

threshold.config - rate_filter cua SID 1001002

```
# Noi dung that trong /usr/local/etc/suricata/suricata_12985_em0/threshold.config
rate_filter gen_id 1, sig_id 1001002, track by_src, count 20, seconds 10,
            new_action drop, timeout 15

# Theo doi rieng tung IP nguon (by_src): 20 lan khop trong 10 giay
# thi chuyen sang DROP trong 15 giay.
```

Hình 4.11. Chính sách ngưỡng và liên kết với instance WAN

Giải thích: Hình ghi lại chính sách sẽ được dùng trong so sánh đối chứng. **Nhận xét:** Hai IP được theo dõi riêng nên cần bằng chứng kích hoạt của cả hai. **Đánh giá:** Cấu hình có mặt chưa phải kết quả phát hiện; Chương 5 phải đối chiếu action, nguồn, thời gian và log backend.

4.6.5. Nhật ký và phạm vi chặn

Bật phần ghi cảnh báo cần thiết, đồng thời giữ EVE JSON nếu dùng để phân tích chi tiết. Trang Alerts hỗ trợ quan sát nhanh; EVE hỗ trợ lọc theo SID, nguồn và thời gian [14]. Đối với phép thử chặn, đối chiếu trạng thái blocked/drop phù hợp với phiên bản cùng hành vi tại Kali và backend. Không mặc định tab Blocked kiểu chặn IP của Legacy Mode là nguồn bằng chứng duy nhất cho Inline IPS.

4.7. Cấu hình JMeter trên hai Kali

4.7.1. Chuẩn bị công cụ và kế hoạch

Kiểm tra Java bằng `java -version` và JMeter bằng `jmeter -v`. Nếu chưa có bản phù hợp, tải gói nhị phân Apache JMeter từ trang chính thức, giải nén và chạy tệp `bin/jmeter` theo hướng dẫn cài đặt [13]. Dùng phiên bản Java được bản JMeter đó hỗ trợ. Khi gặp lỗi truy cập module Java của một bản JMeter cũ, ưu tiên kiểm tra cặp phiên bản và cập nhật bản phù hợp trước khi áp dụng các tùy chọn mở module.

Trên cả hai Kali, tạo Test Plan với một Thread Group và một HTTP Request. Đích là IP frontend, không phải địa chỉ của một backend. Tạo User Defined Variables gồm RUN_ID và SRC; đặt SRC là K1 hoặc K2. Ở HTTP Request, dùng Parameters để thêm run=${RUN_ID} và src=${SRC}. Cách đặt mã giúp tìm đúng lần chạy trong log ứng dụng. Cả hai máy chờ cùng mốc thời gian T0 trước khi gửi lô đầu tiên để bảo đảm khoảng chồng lấn thực tế.

Bảng 4.1. Bộ tham số kiểm chứng đề xuất cho hai Kali

Tham số	Kiểm tra truy cập bình thường	Tải dùng đối chứng chính sách
Threads trên mỗi Kali	1	2
Loop Count	5	20
Ramp-up	1 giây	1 giây
Constant Timer	1000 ms	100 ms
HTTP Request mỗi vòng	1 GET tới /load-test.txt	1 GET tới /load-test.txt
Đích	http, 10.10.10.1, port 8080	Giữ nguyên
Connect / Response timeout	3000 / 5000 ms	Giữ nguyên
Mẫu dự kiến mỗi nguồn	5	40
Tổng dự kiến hai nguồn	10	80

Các tham số ở Bảng 4.1 là cấu hình thực tế dùng cho KB1 và KB3/KB4; mọi lần chạy giữ nguyên giá trị, chỉ thay RUN_ID và SRC. Không bật vòng lặp vô hạn; khi phải dừng vì tài nguyên, ghi số mẫu thực sự hoàn thành.

4.7.2. Cấu hình HTTP Request và ghi kết quả

HTTP Request dùng Protocol `http`, Server Name or IP `10.10.10.1`, Port `8080`, Method `GET`, Path `/load-test.txt`. Để đếm đơn giản, không thêm yêu cầu tải embedded resources hoặc sampler khác vào kế hoạch này. Dùng Response Assertion kiểm tra mã HTTP 200; như vậy kết quả thành công không chỉ phụ thuộc việc kết nối có trả bất kỳ phản hồi nào.

Trong lần kiểm tra nhỏ, dùng View Results Tree để xem request/response và Summary Report hoặc Aggregate Report để xem tổng mẫu, lỗi, độ trễ. Trước lần chạy mới, lưu kết quả cũ rồi chọn Run → Clear All để xóa phần hiển thị. Thao tác này không có nghĩa các tệp JTL cũ đã được xóa. Dùng tên tệp mới cho từng Kali và từng lần chạy để không trộn dữ liệu.

Khi chạy bằng dòng lệnh, có thể dùng dạng lệnh sau sau khi đã lưu kế hoạch đúng tham số. Đổi tên tệp cho Kali 2 và mã kịch bản tương ứng; không chạy đồng thời thêm một bản GUI của cùng kế hoạch ngoài dự kiến.

```
TZ=UTC jmeter -n -t lab-http.jmx -Jrun=KB3_R2 -Jsrc=K1 -Jthreads=2 -Jloops=20 -Jtimer=100 -l
KB3_R2_K1.jtl -j KB3_R2_K1.log
```

lab-http.jmx - tham so hoa bang -J

```
Thread Group: Threads = ${__P(threads,2)}; Loop = ${__P(loops,20)}
HTTP Request: host=${__P(host,10.10.10.1)} port=${__P(port,8080)}
  path=${__P(path,/load-test.txt)}?run=${__P(run,KB3_R1)}&src=${__P(src,K1)}
Constant Timer: ${__P(timer,100)} ms; Timeout: 5000 ms

$ TZ=UTC /home/kali/apache-jmeter-5.6.3/bin/jmeter -n -t lab-http.jmx \
    -Jrun=KB3_R1 -Jsrc=K1 -Jthreads=2 -Jloops=20 -Jtimer=100 \
    -Jpath=/load-test.txt -l KB3_R1_K1.jtl -j KB3_R1_K1.log
$ ... (Kali 2: -Jsrc=K2 -l KB3_R1_K2.jtl)
```

Hình 4.12. Cấu hình JMeter tương ứng trên hai Kali

Giải thích: Hai nguồn dùng cùng cấu hình tải để phép đối chứng có thể so sánh. **Nhận xét:** Ramp-up phân bố thời điểm khởi tạo luồng, không bảo đảm tốc độ HTTP cố định. **Đánh giá:** Tổng mẫu, thời gian và tốc độ sau chạy vẫn phải lấy từ kết quả thực tế.

4.8. Xác nhận hệ thống trước thực nghiệm

Bảng 4.2. Điều kiện sẵn sàng và bằng chứng xác nhận

Điều kiện	Cách xác nhận	Kết quả ghi nhận
Đủ hai nguồn, không trùng IP	ip -br addr trên K1 và K2	Kali 1: 10.10.10.10/24; Kali 2: 10.10.10.11/24; không trùng địa chỉ.
Backend HTTP hoạt động	Truy cập /health.txt từ phía pfSense	web1 và web2 trả /health.txt với HTTP 200 khi kiểm tra từ pfSense.

Điều kiện	Cách xác nhận	Kết quả ghi nhận
HAProxy có hai server UP	Stats và LastChk	Cả hai server UP, LastChk L7OK/200 trên HAProxy Stats.
Đường đi đúng frontend	URL, cấu hình NAT và log proxy/backend	curl tới 10.10.10.1:8080 trả 200; log backend có run=KB1_R1 và src=K1/K2.
Suricata đang ở WAN inline	Settings và log khởi động	Suricata WAN chạy Inline IPS trên netmap; instance hoạt động sau khi nạp.
Hai SID không xung đột	Nội dung luật và kết quả nạp	SID 1001001 và 1001002 nạp không lỗi, không xung đột SID.
Hai Kali được chính sách bao phủ	Alias firewall và danh sách nguồn trong luật	Alias KALI_LAB chứa 10.10.10.10 và 10.10.10.11; luật WAN dùng nguồn KALI_LAB.
Tài nguyên và đồng hồ ổn định	CPU, RAM và thời gian của các VM	CPU/RAM ổn định; pfSense chậm hơn Kali khoảng 2–4 phút, đã chuẩn hóa UTC khi đối chiếu.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

5.1. Mục tiêu và quy trình thực nghiệm chung

Các phép thử được tổ chức từ chức năng đơn lẻ tới phối hợp toàn hệ thống. Trước tiên cần biết truy cập bình thường thành công. Tiếp theo, phép chặn URI xác nhận Suricata có khả năng can thiệp. Chỉ sau hai bước này mới so sánh tải từ hai nguồn khi tắt/bật chính sách theo ngưỡng. Phép thử lỗi backend được tách riêng để không lẫn tác động của health check với tác động của IPS.

Bảng 5.1. Ma trận các kịch bản

Kịch bản	Suricata và chính sách	Backend	Câu hỏi kiểm chứng
KB1 – Bình thường	Inline hoạt động; tải dưới ngưỡng	Hai server UP	Hai Kali có truy cập đúng dịch vụ không?
KB2 – Kiểm tra IPS	Đối chứng instance WAN tắt/bật; dùng SID 1001001	Hai server UP	Luật URI có thực sự chặn không?
KB3 – Chưa chặn theo ngưỡng	Inline và luật ALERT hoạt động; không áp dụng rate_filter	Hai server UP	Hai nguồn có đồng thời tạo tải và tải được phân phối ra sao?
KB4 – Có chặn theo ngưỡng	Giữ KB3, thêm rate_filter của SID 1001002	Hai server UP	Hành động có chuyển sang chặn và lượng tới backend thay đổi thế nào?
KB5 – Lỗi backend	Giữ đường kiểm tra ổn định; tạm không áp dụng rate_filter	Web 1 dừng rồi khôi phục; Web 2 hoạt động	HAProxy có loại và đưa backend trở lại không?

Mỗi lần chạy có một mã riêng, chẳng hạn KB3_R1. Trước khi bắt đầu, nhóm ghi cấu hình hiệu lực, kiểm tra đồng hồ, lưu bộ đếm HAProxy và mở log trên hai backend. Sau khi chạy, nhóm lưu JTL, ảnh và log gắn đúng mã. Kết luận chỉ sử dụng dữ liệu cùng một lần chạy hoặc dữ liệu đối chứng được đánh dấu rõ.

Các phần sau trình bày kết quả đã đo của từng kịch bản, đối chiếu giữa JTL, log backend và sự kiện Suricata; mọi số liệu đều gắn với mã lần chạy tương ứng.

5.2. Kịch bản 1 – Truy cập HTTP bình thường

5.2.1. Mục tiêu và điều kiện

Kịch bản kiểm tra khả năng truy cập dịch vụ của cả hai Kali trước khi tạo lưu lượng đồng thời ở mức cao hơn. Điều kiện ban đầu là HAProxy chạy, hai backend UP, Suricata WAN hoạt động và không còn trạng thái chặn từ lần trước. Dùng trang gốc để kiểm tra đường đi, sau đó dùng cấu hình tải thấp ở Bảng 4.1 để kiểm tra URI `/load-test.txt` dưới ngưỡng của từng nguồn.

5.2.2. Trình tự thực hiện

Bước 1: Trên Kali 1 và Kali 2, ghi lại IP nguồn và gửi một yêu cầu tới frontend. Thay K1 bằng K2 ở máy thứ hai.

```
curl --noproxy '*' -i --max-time 5 "http://10.10.10.1:8080/?case=KB1_K1"
```

Bước 2: Kiểm tra mã HTTP và nội dung SERVER 1 hoặc SERVER 2. Trong cửa sổ Python của backend được chọn, tìm đúng query `case=KB1_K1` hoặc `case=KB1_K2`. **Bước 3:** Chạy kế hoạch tải thấp trên cả hai Kali, lưu JTL riêng và kiểm tra không có DROP tương ứng với lưu lượng dưới ngưỡng.

KB1_R1 - curl tu hai Kali toi frontend

```
KALI 1 - $ curl --noproxy '*' -i --max-time 5 'http://10.10.10.1:8080/?case=KB1_K1'
SRC=10.10.10.10
eth0             UP             10.10.10.10/24
HTTP/1.0 200 OK
server: SimpleHTTP/0.6 Python/3.12.3
date: Mon, 09 Nov 2026 10:52:47 GMT
content-type: text/html
content-length: 153
last-modified: Mon, 09 Nov 2026 08:12:29 GMT
connection: keep-alive

<html><head><title>ATT DDoS Lab</title></head><body>
<h1>ATT DDoS LAB - WEB SERVER</h1>
<h2>SERVER 1</h2>
<p>Host: web1 (10.10.20.20)</p>
</body></html>

KALI 2 - cung lenh, case=KB1_K2
SRC=10.10.10.11
eth0             UP             10.10.10.11/24
HTTP/1.0 200 OK
server: SimpleHTTP/0.6 Python/3.12.3
date: Mon, 09 Nov 2026 10:52:47 GMT
content-type: text/html
content-length: 153
last-modified: Mon, 09 Nov 2026 08:12:39 GMT
connection: keep-alive

<html><head><title>ATT DDoS Lab</title></head><body>
<h1>ATT DDoS LAB - WEB SERVER</h1>
<h2>SERVER 2</h2>
<p>Host: web2 (10.10.20.21)</p>
</body></html>
```

Hình 5.1. Phản hồi truy cập bình thường từ hai Kali

Giải thích: Phản hồi thành công chứng minh yêu cầu đã đi tới một backend qua điểm truy cập frontend. **Nhận xét:** Nhãn SERVER 1 hoặc SERVER 2 xác định máy phục vụ; một yêu cầu riêng lẻ không chứng minh thuật toán phân phối đều. **Đánh giá:** Chỉ khi cả .10 và .11 đều thành công mới xác nhận đường truy cập cho đủ hai nguồn.

KB1_R1 - log backend va Summary Report

```
WEB 1 /var/log/syslog (loc KB1_R1, da bo /health.txt)
Nov 09 10:52:52 web1 python3[738]: 10.10.20.1 - - [09/Nov/2026 10:52:52] "GET /load-test.txt?run=KB1_R1&src=K2 HTTP/1.1" 200 -
Nov 09 10:52:53 web1 python3[738]: 10.10.20.1 - - [09/Nov/2026 10:52:53] "GET /load-test.txt?run=KB1_R1&src=K2 HTTP/1.1" 200 -
Nov 09 10:52:54 web1 python3[738]: 10.10.20.1 - - [09/Nov/2026 10:52:54] "GET /load-test.txt?run=KB1_R1&src=K2 HTTP/1.1" 200 -

WEB 2 /var/log/syslog
Nov 09 10:52:52 web2 python3[736]: 10.10.20.1 - - [09/Nov/2026 10:52:52] "GET /load-test.txt?run=KB1_R1&src=K1 HTTP/1.1" 200 -
Nov 09 10:52:53 web2 python3[736]: 10.10.20.1 - - [09/Nov/2026 10:52:53] "GET /load-test.txt?run=KB1_R1&src=K1 HTTP/1.1" 200 -
Nov 09 10:52:54 web2 python3[736]: 10.10.20.1 - - [09/Nov/2026 10:52:54] "GET /load-test.txt?run=KB1_R1&src=K1 HTTP/1.1" 200 -

KALI 1 - JMeter summary
2026-11-09 10:52:51,569 INFO o.a.j.r.Summariser: summary =      5 in 00:00:06 =    0.9/s Avg:    28 Min:    13 Max:    84 Err:
    0 (0.00%)
KALI 2 - JMeter summary
2026-11-09 10:52:47,394 INFO o.a.j.r.Summariser: summary =      5 in 00:00:06 =    0.9/s Avg:    26 Min:    13 Max:    64 Err:
    0 (0.00%)
```

Hình 5.2. Log backend tương ứng và kết quả tải thấp

Giải thích: Dòng log là bằng chứng ứng dụng đã nhận yêu cầu, bổ sung cho phản hồi tại client. **Nhận xét:** Địa chỉ 10.10.20.1 trong log phù hợp với kết nối do HAProxy mở; các dòng /health.txt không được tính. **Đánh giá:** Nếu cả hai nguồn có lỗi ở mức tải này, cần khắc phục điều kiện nền trước khi đánh giá chính sách ngưỡng.

5.2.3. Kết quả và đánh giá

Kết quả cho lần chạy: Kali 1 hoàn thành 5 mẫu, lỗi 0; Kali 2 hoàn thành 5 mẫu, lỗi 0. Hai backend ghi 5 yêu cầu /load-test.txt đúng mã mỗi server, tương ứng 10 yêu cầu tải; ngoài ra mỗi server có một yêu cầu trang gốc. Trạng thái kiểm chứng của KB1: ĐẠT.

Tiêu chí đạt là cả hai nguồn nhận phản hồi đúng và không có bằng chứng chặn ngoài dự kiến trong phép thử tải thấp. Một kết quả đạt cho thấy cấu hình cơ bản cho phép phục vụ truy cập thử nghiệm; chưa đủ kết luận chính sách không bao giờ chặn nhầm đối với người dùng thực.

5.3. Kịch bản 2 – Đối chứng khả năng chặn của IPS

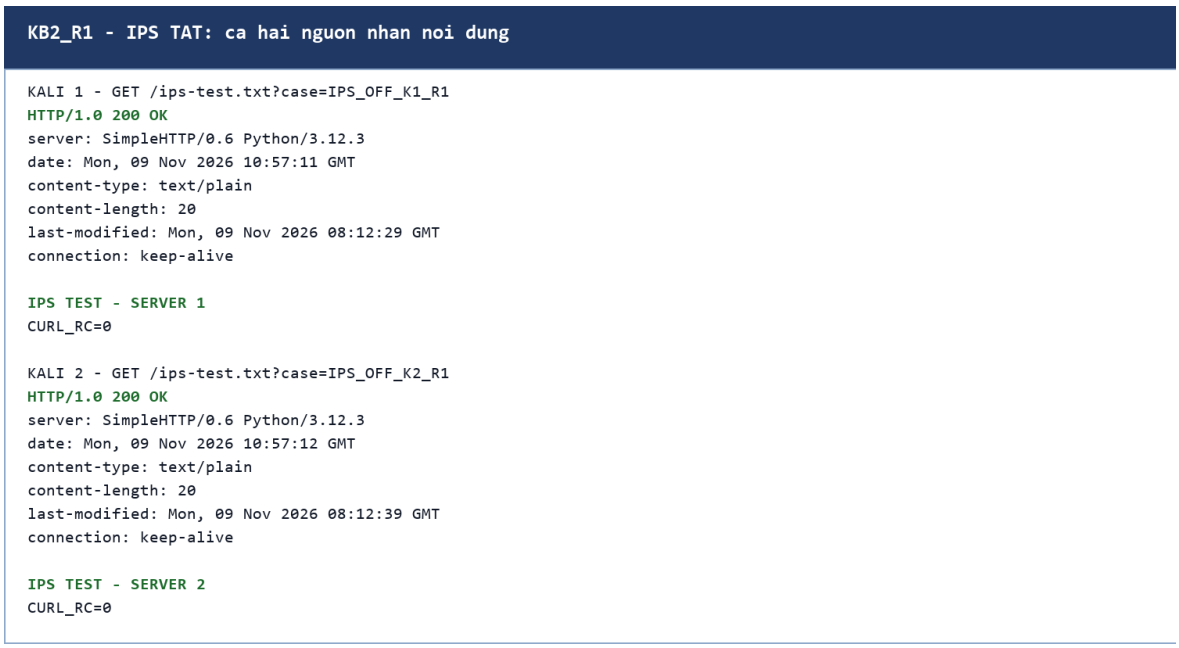
5.3.1. Mục tiêu và biến kiểm soát

Kịch bản dùng SID 1001001 để kiểm tra trực tiếp hành động chặn, giữ nguyên URL đích, cổng, timeout và hai backend. Trạng thái đối chứng thứ nhất tắt instance Suricata WAN; trạng thái thứ hai bật lại đúng instance và luật DROP. Không thay đổi đồng thời NAT, HAProxy hoặc dịch vụ Python trong phép so sánh.

5.3.2. Trình tự khi IPS tắt

Bước 1: Dừng các lần tạo tải và xác nhận hai backend UP. Tạm dừng Suricata trên WAN. **Bước 2:** Từ Kali 1 gửi yêu cầu sau; lặp lại trên Kali 2 với nhãn khác. **Bước 3:** Ghi lại phản hồi và tìm đúng nhãn trong log backend.

```
curl --noproxy '*' -v --max-time 5 "http://10.10.10.1:8080/ips-test.txt?case=IPS_OFF_K1_R1"
```



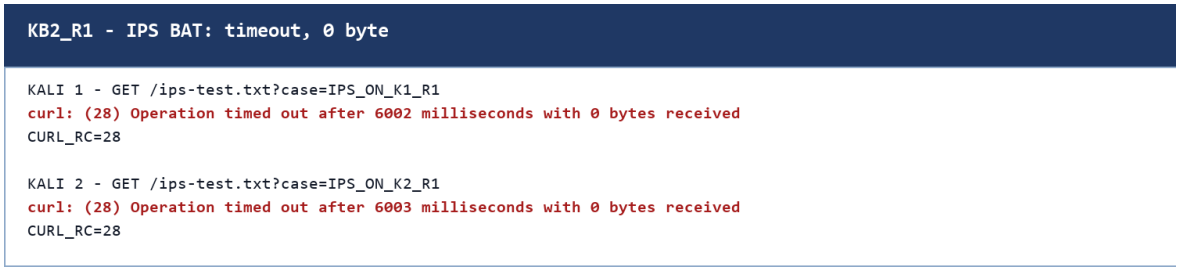
Hình 5.3. Yêu cầu nhận được nội dung khi IPS tắt

Giải thích: Nội dung IPS TEST - SERVER 1 xác nhận file tồn tại và server trả về khi instance WAN đã dừng. Nhận xét: Cả hai Kali đều nhận HTTP 200 với nội dung SERVER tương ứng; log backend có đúng dòng IPS_OFF. Đánh giá: Đây là đối chứng "dịch vụ truy cập được" trước khi bật IPS.

5.3.3. Trình tự khi IPS bật

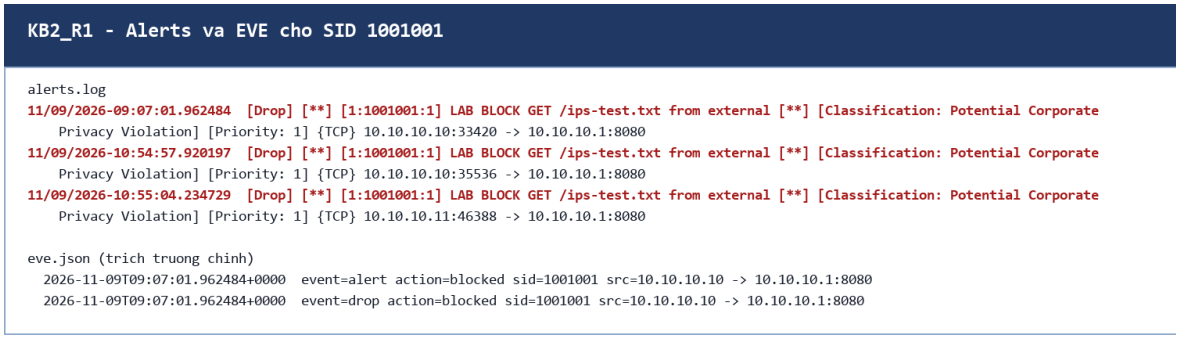
Bước 4: Bật lại Suricata WAN, xác nhận instance chạy và luật SID 1001001 đã nạp. **Bước 5:** Tạo kết nối mới từ mỗi Kali bằng URL có nhãn IPS_ON. **Bước 6:** Đối chiếu đồng thời phản hồi client, action trong Alerts/EVE và log hai backend.

```
curl --noproxy '*' -v --max-time 5 "http://10.10.10.1:8080/ips-test.txt?case=IPS_ON_K1_R1"
```



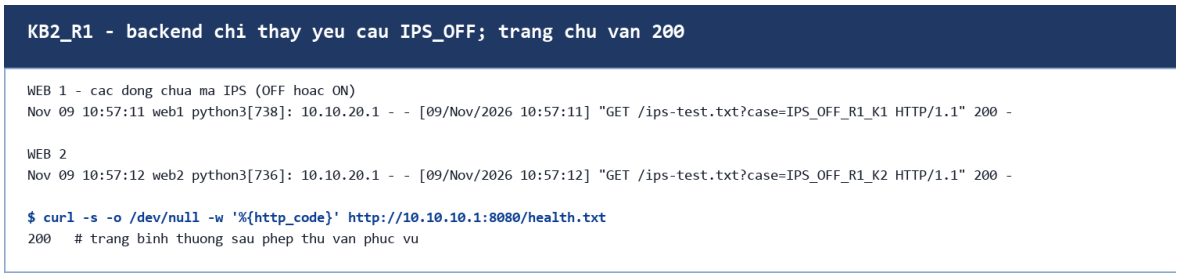
Hình 5.4. Kali không nhận nội dung khi luật chặn URI có hiệu lực

Giải thích: curl báo timeout sau khoảng 6 giây và nhận 0 byte. Nhận xét: Bắt tay TCP đã thành công trước khi yêu cầu HTTP bị ảnh hưởng là điều có thể xảy ra khi luật kiểm tra URI. Đánh giá: Timeout riêng lẻ chưa chứng minh Suricata là nguyên nhân; phải ghép với sự kiện chặn cùng nguồn và đích.



Hình 5.5. Suricata ghi nhận hành động chặn SID 1001001

Giải thích: Ảnh Alerts ghi nhận các dòng 1:1001001 với mô tả LAB IPS block test URI cho cả nguồn 10.10.10.10 và 10.10.10.11, đích 10.10.10.1:8080. Nhận xét: Kết quả này phù hợp với timeout tại cả hai Kali trong phép kiểm tra URI. Đánh giá: Có cơ sở xác nhận Suricata đã thực thi chặn đối với cả hai nguồn ở phép thử đó.



Hình 5.6. Đối chiếu log backend và yêu cầu bình thường sau phép chặn

Giải thích: Việc không thấy mã IPS_ON tại cả hai ứng dụng phù hợp với khả năng yêu cầu đã bị chặn trước backend. **Nhận xét:** Một ảnh chỉ hiển thị vài dòng cuối không chứng minh toàn bộ log không chứa mã đó; cần tìm trong log đầy đủ của khoảng thời gian. **Đánh giá:** Kết hợp hành động chặn, không nhận nội dung và trang bình thường vẫn phục vụ cho thấy phép kiểm tra có tính chọn lọc đối với URI, chưa chứng minh nhận diện DDoS theo cường độ.

5.3.4. Kết luận kịch bản

Minh chứng hiện có xác nhận chức năng chặn URI trên đường vào HAProxy đối với cả hai nguồn. Khi IPS tắt, Kali 1 và Kali 2 đều nhận HTTP 200 với nội dung IPS TEST - SERVER 1 và SERVER 2. Khi IPS bật, cả hai đều timeout sau khoảng 6 giây, nhận 0 byte; Suricata ghi 6 sự kiện chặn SID 1001001 cho .10 và .11. Trang bình thường sau phép thử trả HTTP 200. Không dùng số lần xuất hiện SID 1001001 để đánh giá hiệu quả chính sách rate_filter của SID 1001002.

5.4. Kịch bản 3 – Hai nguồn đồng thời khi chưa chặn theo ngưỡng

5.4.1. Mục tiêu và điều kiện

Kịch bản xác định lưu lượng đầu vào, phản hồi và cách HAProxy phân phối khi chưa chuyển hành động theo ngưỡng. Suricata vẫn hoạt động inline và luật ALERT 1001002 vẫn được giữ; chỉ chưa áp dụng rate_filter. Cách bố trí này giữ chi phí kiểm tra của Suricata tương đối nhất quán giữa KB3

và KB4. Luật URI 1001001 không khớp đường dẫn `/load-test.txt` nên không dùng để giải thích kết quả của kịch bản này.

Hai backend phải UP và cùng trọng số. Mỗi Kali sử dụng bộ tham số tải đối chứng ở Bảng 4.1. KB3 được chạy hai lần với mã KB3_R1 và KB3_R2; kết quả định lượng trong báo cáo lấy từ KB3_R2 vì lần này có đủ JTL, log và ảnh Stats trước/sau. Bộ tham số được lựa chọn để kiểm chứng ngưỡng nhỏ, không nhằm khẳng định đã làm cạn năng lực máy chủ.

5.4.2. Các bước tiến hành

Bước 1: Ghi ảnh Stats ban đầu, xác nhận không còn tải cũ và mở log backend. Bước 2: Lưu kết quả JMeter cũ, Clear All phần hiển thị và dùng tệp JTL mới. Bước 3: Khởi chạy kế hoạch trên K1 và K2 gần cùng thời điểm; ghi thời gian bắt đầu và kết thúc từ dữ liệu gốc. Bước 4: Khi hai nguồn kết thúc, chụp kết quả, Stats và log hai server. Bước 5: Lọc đúng mã KB3_R2, đếm số yêu cầu ở từng backend và kiểm tra khoảng chạy đồng thời.

KB3 - hai tien trinh JMeter chay dong thoi (KB3_R2)

```
KALI 1 (10.10.10.10)
$ TZ=UTC jmeter -n -t lab-http.jmx -Jrun=KB3_R2 -Jsrc=K1 -Jthreads=2 -Jloops=20 -Jtimer=100 -Jpath=/load-test.txt -l KB3_R2_K1.jtl
2026-11-09 11:30:28,090 INFO o.a.j.r.Summariser: summary =     40 in 00:00:04 =   10.8/s Avg:    40 Min:    21 Max:   189 Err:     0 (0.00%)
  bat dau 11:30:25.231  ket thuc 11:30:28.076  (UTC)

KALI 2 (10.10.10.11)
$ TZ=UTC jmeter -n -t lab-http.jmx -Jrun=KB3_R2 -Jsrc=K2 -Jthreads=2 -Jloops=20 -Jtimer=100 -Jpath=/load-test.txt -l KB3_R2_K2.jtl
2026-11-09 11:30:26,987 INFO o.a.j.r.Summariser: summary =     40 in 00:00:04 =   10.5/s Avg:    39 Min:    21 Max:   269 Err:     0 (0.00%)
  bat dau 11:30:23.895  ket thuc 11:30:26.972  (UTC)

# Hai tien trinh dung chung mot moc rao 35 giay (barrier) nen chay dong thoi:
# cua so chong lan 11:30:25.231 - 11:30:26.972 (~1,7 giay).
```

Hình 5.7. Hai Kali cùng chạy kế hoạch gửi tới frontend

Giải thích: Hình thể hiện hai tiến trình gửi yêu cầu từ hai IP khác nhau. **Nhận xét:** Số luồng đang hoạt động trên màn hình chỉ phản ánh thời điểm chụp; khoảng đồng thời được xác nhận bằng JTL. **Đánh giá:** Nếu không có thời gian chồng lấn, cần mô tả lần chạy là hai lượt riêng và thực hiện lại trước khi sử dụng như minh chứng phân tán.

KB3 - Summary Report tren tung nguon (KB3_R2)

```
KALI 1
2026-11-09 11:30:28,090 INFO o.a.j.r.Summariser: summary =     40 in 00:00:04 =   10.8/s Avg:    40 Min:    21 Max:   189 Err:
    0 (0.00%)
KALI 2
2026-11-09 11:30:26,987 INFO o.a.j.r.Summariser: summary =     40 in 00:00:04 =   10.5/s Avg:    39 Min:    21 Max:   269 Err:
    0 (0.00%)

Tong thuc te: 80 mau; loi: 0; Avg 40/39 ms; P95 44/47 ms; 10,8/10,5 req/s.
```

Hình 5.8. Kết quả JMeter của KB3 trên từng nguồn

Giải thích: JTL của KB3_R2 ghi 40 mẫu ở mỗi Kali, tổng 80 mẫu, 0 lỗi; thời gian trung bình 41,0 ms và 39,3 ms. Nhận xét: Mỗi tệp JTL chỉ đại diện một nguồn; tổng 80 mẫu được xác nhận bằng hai tệp. Đánh giá: Với 0 lỗi và thời gian ổn định, dịch vụ đáp ứng được mức tải đã thử; chưa chứng minh hệ thống chịu được tải lớn hơn.

```
KB3 - /load-test.txt tai ca hai backend (KB3_R2)

WEB 1 - 42 dong (2 probe + 40 yeu cau tai); theo src: K1=20, K2=20
10.10.20.1 - - [09/Nov/2026 11:27:05] "GET /load-test.txt?case=KB3_R2_PROBE_K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:29:46] "GET /load-test.txt?case=KB3_R2_PROBE_K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:30] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:30] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:31] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:31] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
  ...
WEB 2 - 42 dong (2 probe + 40 yeu cau tai); theo src: K1=20, K2=20
10.10.20.1 - - [09/Nov/2026 11:27:06] "GET /load-test.txt?case=KB3_R2_PROBE_K2 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:29:46] "GET /load-test.txt?case=KB3_R2_PROBE_K2 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:30] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:31] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:31] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:30:31] "GET /load-test.txt?run=KB3_R2&src=K1 HTTP/1.1" 200 -

# Moi dong deu co run=KB3_R2 va src=K1/K2; /health.txt da bi loai.
```

Hình 5.9. Yêu cầu thử nghiệm được nhận ở cả hai backend

Giải thích: Log của cả hai backend đều có GET /load-test.txt với mã 200, gắn run=KB3_R2 và nhãn src=K1/K2. Nhận xét: Số dòng tải là 40 ở mỗi backend, cộng hai dòng kiểm tra; điều này chứng minh cả hai backend tham gia phục vụ. Đánh giá: Kết hợp JTL 40 mẫu mỗi nguồn cho thấy HAProxy phân phối cân bằng trong lần chạy.

```
KB3 - HAProxy Stats truoc / sau (KB3_R2)

Truoc (11:29Z): web1 stot=1, web2 stot=1, backend=2
Sau   (11:31Z): web1 stot=40, web2 stot=40, backend=80
Backend row: web1   UP    L7OK          1 -> 40       +39
Backend row: web2   UP    L7OK          1 -> 40       +39

# Delta: web1 +39, web2 +39; log backend ghi 80 yeu cau tai (40/40), khong tinh health check.
```

Hình 5.10. Đối chiếu HAProxy Stats trước và sau KB3

Giải thích: Ảnh Stats trước và sau KB3_R2 cho thấy hai server UP, LastChk L7OK/200; Sessions Total tăng từ 1 lên 40 ở cả web1 và web2. Nhận xét: Mức tăng bằng nhau hỗ trợ nhận định HAProxy phân phối đều 40 yêu cầu của mỗi nguồn cho hai backend. Đánh giá: Sessions Total là bộ đếm phiên, không đồng nhất với số HTTP Request; kết luận định lượng dùng log lọc theo mã lần chạy.

5.4.3. Kết quả định lượng và đánh giá

Bảng 5.2. Kết quả của một lần chạy hai nguồn không áp dụng rate_filter

Đại lượng	Kali 1	Kali 2	Tổng hoặc cách đối chiếu
Mẫu dự kiến	40	40	80 (giữ Bảng 4.1)
Mẫu thực tế	40	40	N1 + N2 = 80
Số mẫu lỗi	0	0	E1 + E2 = 0
Bắt đầu / kết thúc	11:30:25,231 – 11:30:28,043	11:30:23,895 – 11:30:26,945	Chồng lấn 1,714 giây
Thời gian TB / P95	TB 41,0 ms; P95 44 ms	TB 39,3 ms; P95 47 ms	Tính từ 80 mẫu gốc; P95 44/47 ms

Đại lượng	Kali 1	Kali 2	Tổng hoặc cách đối chiếu
Yêu cầu tại web1 / web2	Không tách bằng IP TCP backend	Lọc bằng src trong URI	42 / 42 (40 yêu cầu tải và 2 yêu cầu kiểm tra mỗi server)

Mức phân phối của lần chạy được tính từ log đã lọc: web1 50%, web2 50% (42/42 dòng, gồm 40 yêu cầu tải và 2 yêu cầu kiểm tra mỗi server). Nếu cả hai cùng nhận yêu cầu và phản hồi hợp lệ, chức năng phân phối tải đã được kiểm chứng. Muốn kết luận hai backend làm tăng năng lực so với một backend, cần thêm phép đối chứng một server và hai server dưới cùng điều kiện; bộ số liệu hiện tại không tự cung cấp so sánh đó.

5.5. Kịch bản 4 – Hai nguồn đồng thời khi bật chính sách ngưỡng

5.5.1. Mục tiêu và điều kiện

Mục tiêu là kiểm tra SID 1001002 có chuyển sang hành động chặn theo chính sách hay không và xác định ảnh hưởng tới lượng yêu cầu đến backend. Giữ cấu hình như KB3, bật rate_filter tại WAN và dùng mã mới KB4_R2. Không thay đổi số luồng hoặc giảm số backend đồng thời với bật chính sách, vì sẽ không tách được nguyên nhân của thay đổi kết quả.

5.5.2. Các bước tiến hành

Bước 1: Xác nhận chính sách đã nạp, hai backend UP và các trạng thái cũ đã hết ảnh hưởng. Bước 2: Ghi Stats ban đầu, dùng JTL mới và khởi chạy cùng cấu hình trên hai Kali. Bước 3: Quan sát cảnh báo theo từng nguồn; chụp sự kiện có SID 1001002 và hành động thực tế. Bước 4: Chạy một yêu cầu đối chứng nhẹ tới / bằng kết nối mới, ghi thời điểm và phản hồi. Bước 5: Lưu dữ liệu cuối, lọc log backend theo KB4_R2 và so sánh với KB3_R2 có điều kiện đầu vào tương đương.

```
KB4 - Summary Report voi rate_filter bat (KB4_R2)

KALI 1
2026-11-09 11:34:53,667 INFO o.a.j.r.Summariser: summary =     40 in 00:00:14 =    2.9/s Avg:   536 Min:    19 Max:  5040 Err:
    4 (10.00%)
KALI 2
2026-11-09 11:34:53,845 INFO o.a.j.r.Summariser: summary =     40 in 00:00:14 =    2.9/s Avg:   540 Min:    22 Max:  5204 Err:
    4 (10.00%)

Moi nguon: 36 x HTTP 200 + 4 x SocketTimeoutException (10%); Max 5040 / 5204 ms.
P95 cua mau thanh cong chi con 50 / 40 ms; P95 toan bo 5008 / 5021 ms.
```

Hình 5.11. Kết quả JMeter khi chính sách ngưỡng được bật

Giải thích: Lỗi ở phía nguồn có thể là phản ứng mong đợi khi yêu cầu bị IPS chặn. **Nhận xét:** Lỗi tăng không tự chứng minh giải pháp làm dịch vụ kém hơn hoặc tốt hơn; cần biết loại yêu cầu và nguyên nhân. **Đánh giá:** Nếu mẫu bị chặn phải chờ timeout, throughput của JMeter có thể giảm do nguồn chờ phản hồi, nên cần đối chiếu số yêu cầu thực sự tới backend.

```
KB4 - DROP cua SID 1001002 cho ca hai nguon (KB4_R2)

alerts.log - 6 dong [Drop] trong cua so tai KB4_R2
11/09/2026-11:30:44.289754  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.11:46838 -> 10.10.10.1:8080
11/09/2026-11:30:44.385061  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.11:46822 -> 10.10.10.1:8080
11/09/2026-11:30:44.461350  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.10:32828 -> 10.10.10.1:8080
11/09/2026-11:30:44.463742  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.10:32838 -> 10.10.10.1:8080
11/09/2026-11:30:49.638303  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.11:59930 -> 10.10.10.1:8080
11/09/2026-11:30:49.642331  [Drop] [**] [1:1001002:1] LAB ALERT /load-test.txt high rate [**] [Classification: Potential Corporate Privacy
    Violation] [Priority: 1] {TCP} 10.10.10.11:59936 -> 10.10.10.1:8080

# Drop theo nguon: Kali 1 = 2, Kali 2 = 4. Dong ho pfSense cham ~4 phut so voi Kali
# (gio tren dong log la gio pfSense; moc gio thuc lay tu JTL cua hai may Kali).
```

Hình 5.12. Bằng chứng chặn theo ngưỡng của cả hai IP nguồn

Giải thích: Sự kiện của SID 1001002 là bằng chứng đúng loại để xem xét chính sách ngưỡng. **Nhận xét:** Dòng ALERT trước ngưỡng có thể xuất hiện cùng dòng DROP sau khi chính sách kích hoạt; chúng có ý nghĩa khác nhau. **Đánh giá:** Nếu chỉ .10 bị chặn, chưa thể kết luận chính sách đã hoạt động với .11; cần kiểm tra luật, tốc độ thực tế và thời gian theo dõi của nguồn còn lại.

```
KB4 - yeu cau con lai van toi backend (KB4_R2)

WEB 1 - 37 dong KB4_R2 (1 probe + 36 yeu cau tai; src K1=18, K2=18)
10.10.20.1 - - [09/Nov/2026 11:34:02] "GET /load-test.txt?case=KB4_R2_PROBE_K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -
  ...
WEB 2 - 37 dong KB4_R2 (1 probe + 36 yeu cau tai; src K1=18, K2=18)
10.10.20.1 - - [09/Nov/2026 11:34:03] "GET /load-test.txt?case=KB4_R2_PROBE_K2 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:34:46] "GET /load-test.txt?run=KB4_R2&src=K1 HTTP/1.1" 200 -

# Tong 72 yeu cau tai toi backend so voi 80 cua KB3: giam (80-72)/80 = 10%.
Yeu cau doi chung: GET /?case=KB4_CONTROL => 200 (ca hai lan).
```

Hình 5.13. Lưu lượng tới backend và yêu cầu đối chứng khi chặn theo ngưỡng

Giải thích: Một số yêu cầu có thể tới backend trước khi điều kiện ngưỡng đạt, vì vậy không đặt tiêu chí bắt buộc log backend phải bằng 0. **Nhận xét:** Yêu cầu đối chứng tới / nằm ngoài URI được theo dõi, chỉ xác nhận dịch vụ vẫn phục vụ loại truy cập đó tại thời điểm kiểm tra. **Đánh giá:** Kết quả chưa chứng minh người dùng hợp lệ truy cập cùng URI và cùng IP với nguồn tải sẽ không bị ảnh hưởng.

5.5.3. Tổng hợp và diễn giải

Bảng 5.3. So sánh hai trạng thái chính sách dưới cùng cấu hình tải

Chỉ tiêu	KB3 chưa áp dụng rate_filter	KB4 có rate_filter	Cách diễn giải
Tổng mẫu thực tế hai Kali	80	80	Cùng cấu hình 2 luồng × 20 vòng mỗi nguồn
Tổng mẫu thành công / lỗi	80 thành công / 0 lỗi	72 thành công / 8 lỗi	KB4 có 4 timeout mỗi nguồn

Chỉ tiêu	KB3 chưa áp dụng rate_filter	KB4 có rate_filter	Cách diễn giải
P95 mẫu thành công	44 ms (K1) / 47 ms (K2)	50 ms (K1) / 40 ms (K2)	Chỉ tính mẫu thành công
Thời gian hoàn tất hai nguồn	≈ 4,2 giây	≈ 13,4 giây	KB4 chậm do mỗi nguồn chờ 4 timeout 5 giây
Yêu cầu tới web1	42	37	Log đúng run và URI; gồm yêu cầu kiểm tra
Yêu cầu tới web2	42	37	Đã loại health check khỏi số liệu tải
DROP SID 1001002 của K1 / K2	0	2 / 4	SID 1001002, action blocked
Yêu cầu đối chứng bình thường	HTTP 200	HTTP 200	Yêu cầu / sau mỗi lần chạy

Gọi B0 và B1 là tổng yêu cầu tải tới hai backend trong KB3 và KB4. Với cùng 80 mẫu đầu vào thực tế, B0 = 80 và B1 = 72, tỷ lệ giảm lượng yêu cầu tới backend là (80 − 72)/80 × 100% = 10,0%. Giá trị này chỉ mô tả mức giảm trong điều kiện lab; không dùng riêng chênh lệch B0 − B1 như hiệu quả chặn tuyệt đối.

Khi dữ liệu cho thấy cả hai nguồn có DROP đúng SID, lượng yêu cầu tới backend giảm và yêu cầu đối chứng vẫn thành công, có thể kết luận chính sách đã thực hiện giảm lưu lượng khớp luật trong điều kiện thử nghiệm. Nếu chỉ có lỗi client mà không có action chặn hoặc backend vốn đã DOWN, chưa đủ căn cứ đưa ra kết luận đó. Trạng thái thực nghiệm KB4 của nhóm: ĐÃ KIỂM CHỨNG; cả hai nguồn đều có sự kiện DROP SID 1001002, lượng yêu cầu tải tới backend giảm 10%, yêu cầu đối chứng trả HTTP 200.

5.6. Kịch bản 5 – Một backend ngừng hoạt động và khôi phục

5.6.1. Mục tiêu và điều kiện

Kịch bản đánh giá health check và khả năng tiếp tục phân phối tới backend còn hoạt động. Dùng các yêu cầu nhẹ, tạm không áp dụng rate_filter để loại yếu tố chặn theo ngưỡng. Giữ Suricata và cấu hình đường đi ổn định. Cả hai backend UP lúc bắt đầu; khi tạo sự cố chỉ dừng tiến trình Python của Web 1, không tắt pfSense hoặc đổi luật firewall.

5.6.2. Trình tự tiến hành

Bước 1: Chụp Stats của hai backend UP và gửi một số yêu cầu có mã KB5_BEFORE. **Bước 2:** Tại cửa sổ Python của Web 1, nhấn Ctrl+C để dừng dịch vụ và ghi thời điểm. **Bước 3:** Theo dõi Stats đến khi web1 chuyển DOWN, trong khi web2 vẫn UP. **Bước 4:** Gửi các yêu cầu mới có mã KB5_DURING tới frontend và xem nội dung phản hồi. **Bước 5:** Khởi động lại Python trên Web 1 bằng lệnh cũ, theo dõi server trở lại UP rồi gửi yêu cầu có mã KB5_AFTER.

```
KB5 - truoc su co: ca hai backend UP (KB5_R1)

HAProxy Stats luc 11:11:20Z
  web1  10.10.20.20:8080  UP  L7OK   stot=5
  web2  10.10.20.21:8080  UP  L7OK   stot=9
  backend web_pool        UP          stot=14

Phan hoi frontend voi 4 yeu cau KB5_BASE (web1 2, web2 2):
  SERVER 2  HTTP/1.0 200 OK  (date 11:11:20 GMT)
  SERVER 1  HTTP/1.0 200 OK  (date 11:11:20 GMT)

# Moc bat dau: 11:11:20-11:11:21Z; ca hai backend cung nhan yeu cau.
```

Hình 5.14. Trạng thái hai backend trước khi tạo sự cố

Giải thích: Đây là mốc nền cho phép thử sự cố. **Nhận xét:** Nếu web1 đã lỗi từ trước, không xác định được độ trễ phát hiện của lần dừng đang thử. **Đánh giá:** Mốc thời gian và trạng thái nền giúp mô tả đúng chuỗi chuyển đổi sau đó.

```
KB5 - Web 1 DOWN, Web 2 tiep tuc phuc vu (KB5_R1)

Dung dich vu:  systemctl stop att-web   luc 11:11:30Z
HAProxy phat hien: web1 DOWN (L4CON) luc 11:11:33Z  (~3 giay)

HAProxy Stats luc 11:11:34Z
  web1  10.10.20.20:8080  DOWN  L4CON   stot=7   lastchg=8 s
  web2  10.10.20.21:8080  UP    L7OK    stot=11

4 yeu cau KB5_DURING qua frontend: 4/4 nhan SERVER 2
HTTP/1.0 200 OK
<h1>ATT DDoS LAB - WEB SERVER</h1>
<h2>SERVER 2</h2>

# Khong yeu cau nao bi loi; web2 phuc vu thay web1.
```

Hình 5.15. Web 1 DOWN trong khi Web 2 tiếp tục phục vụ

Giải thích: HAProxy loại server không đáp ứng khỏi lựa chọn phục vụ sau khi health check đạt điều kiện thất bại. **Nhận xét:** Một số yêu cầu trong khoảng phát hiện vẫn có thể thất bại; cần ghi nhận nếu có. **Đánh giá:** Các yêu cầu mới thành công trên Web 2 chứng minh khả năng duy trì một phần phục vụ khi một backend hỏng, không chứng minh hệ thống hoàn toàn không gián đoạn.

```
KB5 - Web 1 tro lai UP va nhan yeu cau (KB5_R1)

Khoi dong lai:  systemctl start att-web  luc 11:11:54Z
HAProxy bao web1 UP (L7OK/200) luc 11:11:58Z  (~4 giay)

HAProxy Stats luc 11:12:16Z
  web1  UP  L7OK  stot=7     web2  UP  L7OK  stot=15

Log Web 1 (systemd + python3):
10.10.20.1 - - [09/Nov/2026 11:11:20] "GET /?case=KB5_BASE_R1&n=2 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:11:21] "GET /?case=KB5_BASE_R1&n=4 HTTP/1.1" 200 -
Nov 09 11:11:30 web1 systemd[1]: Stopping att-web.service - ATT lab python http server...
Nov 09 11:11:30 web1 systemd[1]: Stopped att-web.service - ATT lab python http server.
Nov 09 11:11:54 web1 systemd[1]: Started att-web.service - ATT lab python http server.
10.10.20.1 - - [09/Nov/2026 11:12:10] "GET /?case=KB5_AFTER_R1&n=2 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:12:12] "GET /?case=KB5_AFTER_R1&n=4 HTTP/1.1" 200 -
10.10.20.1 - - [09/Nov/2026 11:12:13] "GET /?case=KB5_AFTER_R1&n=6 HTTP/1.1" 200 -

# 6 yeu cau KB5_AFTER sau khoi phuc dien ra xen ke web2, web1, web2, web1, web2, web1.
```

Hình 5.16. Web 1 trở lại UP và tham gia nhận yêu cầu

Giải thích: Health check tiếp tục theo dõi server và cho phép đưa backend phục hồi trở lại pool. **Nhận xét:** UP chưa đủ chứng minh đã có yêu cầu người dùng được phân phối, nên cần thêm log

GET sau khôi phục. **Đánh giá:** Chuỗi UP → DOWN → UP và log tương ứng hoàn thiện bằng chứng cho cơ chế phục hồi backend.

5.6.3. Kết quả cần ghi nhận

Bảng 5.4. Kết quả phép thử lỗi backend

Sự kiện hoặc chỉ số	Giá trị thực tế
Thời điểm dừng Python Web 1	11:11:30Z
Thời điểm HAProxy đánh dấu web1 DOWN	11:11:33Z
Độ trễ phát hiện quan sát	≈ 3 giây
Yêu cầu thành công / lỗi trong giai đoạn chuyển đổi	4 thành công / 0 lỗi
Phản hồi qua frontend khi chỉ Web 2 phục vụ	HTTP 200; nội dung SERVER 2 (4/4)
Thời điểm bật lại Python / thời điểm UP	11:11:54Z / 11:11:58Z
Log Web 1 nhận mã KB5_AFTER	CÓ; KB5_AFTER lúc 11:12:10–11:12:13Z

Bảng 5.4 ghi các mốc thời gian quan sát thực tế, không dùng thời gian lý thuyết interval × fall hoặc interval × rise. Độ trễ phát hiện phụ thuộc thời điểm phép kiểm tra, timeout và lập lịch của môi trường ảo; giá trị ghi nhận là khoảng 3 giây cho DOWN và khoảng 4 giây cho UP.

5.7. Tổng hợp mức độ kiểm chứng

Bảng 5.5. Phạm vi kết luận từ các minh chứng

Nội dung	Bằng chứng hiện có hoặc cần bổ sung	Kết luận phù hợp
Chặn URI tại WAN	Timeout 6 giây, 0 byte; 6 sự kiện SID 1001001 action blocked cho .10 và .11	Đã kiểm chứng cho cả hai nguồn
Phân phối tới hai backend	Log /load-test.txt: web1 42, web2 42; Stats UP, Sessions 1→40 ở cả hai	Đã kiểm chứng hai backend cùng tham gia
Hai Kali hoạt động đồng thời	JTL K1/K2 cùng run; chồng lấn 1,714 giây	Đã kiểm chứng hai nguồn chạy đồng thời
Chặn theo ngưỡng của cả hai nguồn	6 sự kiện DROP SID 1001002 (K1 2, K2 4)	Đã kiểm chứng cả hai nguồn

Nội dung	Bằng chứng hiện có hoặc cần bổ sung	Kết luận phù hợp
Mức giảm yêu cầu tới backend	B0 = 80, B1 = 72; giảm 10%	Đã định lượng trong điều kiện lab
Tiếp tục phục vụ khi backend hỏng	Chuỗi UP → DOWN → UP; 4/4 yêu cầu SERVER 2; log KB5_AFTER	Đã kiểm chứng failover và phục hồi

Các bộ đếm Sessions Total được dùng đúng ý nghĩa là số phiên tích lũy. Với KB3_R2, bộ đếm tăng từ 1 lên 40 ở mỗi server; với KB4_R2, tăng từ 0 lên 36. Số yêu cầu tải thực tế tới backend được xác định từ log có mã lần chạy, không lấy trực tiếp từ chênh lệch bộ đếm.

5.8. Đánh giá khả năng phối hợp Suricata và HAProxy

Mô hình phân tách hai nhiệm vụ trong bảo vệ tính sẵn sàng. Suricata kiểm soát lưu lượng khớp luật ở phía nguồn; HAProxy phân phối phần yêu cầu được tiếp nhận tới các backend còn đủ điều kiện. Thử nghiệm chặn URI và hoạt động của hai backend cho thấy hai thành phần có thể cùng tồn tại trên đường truy cập đã thiết kế. Đó là nền tảng để thực hiện so sánh theo ngưỡng.

Về phát hiện, chính sách hiện tại nhận diện một biểu hiện được định nghĩa trước: nguồn lab gửi GET tới một URI với tần suất đạt ngưỡng. Nó không tự xác định động cơ người gửi và không đại diện cho mọi hình thức DDoS. Về giảm thiểu, hành động chặn chỉ có ý nghĩa thực nghiệm khi có bằng chứng giảm yêu cầu tới backend hoặc thay đổi phục vụ được giải thích rõ.

Về cân bằng tải, hai backend cùng xử lý giúp tránh tập trung toàn bộ yêu cầu được chấp nhận vào một server. Tuy nhiên, kết luận "hiệu năng tăng gấp đôi" cần một phép đo năng lực riêng; không thể suy ra từ cấu hình hai server hoặc số phiên gần bằng nhau. Khả năng tiếp tục phục vụ khi một server hỏng cũng phải được xác nhận bằng KB5, thay vì suy từ việc health check đã được bật.

5.9. Hạn chế và yếu tố ảnh hưởng kết quả

Quy mô nguồn và mô hình tải: Hai Kali cung cấp hai nguồn độc lập trong một môi trường có kiểm soát. Các nguồn có thể cùng chạy trên một máy thật nên không tái hiện đường truyền, độ trễ hoặc quy mô của hệ thống phân tán trên Internet. File tĩnh nhỏ cũng không thể hiện đầy đủ chi phí xử lý của ứng dụng động.

Giới hạn chính sách: `track by_src` theo dõi riêng từng IP. Nhiều nguồn dưới ngưỡng vẫn có thể tạo tổng tải lớn. URI lab giúp đo rõ hành vi nhưng giới hạn phạm vi kiểm tra. Hai người dùng sau cùng một IP chia sẻ bộ đếm nguồn có thể chịu ảnh hưởng lẫn nhau; việc đánh giá chặn nhầm cần dữ liệu truy cập hợp lệ phù hợp hơn.

Giới hạn tài nguyên: pfSense chạy đồng thời IPS và reverse proxy nên chi phí xử lý được cộng trên cùng VM. Các lần Kali bị treo hoặc host thiếu RAM là dấu hiệu cần kiểm tra tài nguyên, không được xem tự động là kết quả DDoS vào backend. Mạng Internal Network cũng không đại diện cho băng thông đường WAN thực tế.

Giới hạn bằng chứng: Ảnh chụp thể hiện một thời điểm, còn các bộ đếm có thể tích lũy qua nhiều lần chạy. Không có toàn bộ log và mã lần chạy thì khó xác định mẫu thiếu do bị chặn, do timeout, do thử lại hay do đọc nhầm kết quả cũ. Vì thế, báo cáo giữ dữ liệu gốc và chỉ kết luận trong phạm vi được đối chiếu.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đề tài xây dựng phương án mô phỏng DDoS HTTP theo kiến trúc hai nguồn Kali/JMeter cùng hướng tới một frontend pfSense, phía sau là hai web server Ubuntu. Suricata Inline IPS kiểm tra lưu lượng tại WAN, còn HAProxy lựa chọn backend cho các yêu cầu được tiếp nhận. Thiết kế này cho phép nghiên cứu việc phát hiện, ngăn chặn và phân phối tải trong cùng một luồng xử lý có thể quan sát.

Các kết quả đã ghi nhận gồm khả năng trả nội dung qua frontend, hành động chặn của SID 1001001 đối với URI kiểm tra ở cả hai nguồn và việc cả hai backend nhận yêu cầu HTTP. Sự kết hợp giữa timeout tại Kali, sự kiện chặn tại Suricata và log backend là phương pháp đối chiếu phù hợp hơn việc chỉ nhìn một màn hình lỗi. Tương tự, hai backend UP và có log xử lý hỗ trợ kết luận HAProxy đã phân phối hoạt động, nhưng không tự chứng minh mức tăng hiệu năng.

Kết quả định lượng của KB3_R2 và KB4_R2 được ghi tại Bảng 5.2 và Bảng 5.3; kết quả KB5 tại Bảng 5.4. Tỷ lệ giảm yêu cầu tải tới backend là 10%, P95 của các mẫu thành công trong KB4 là 50 ms và 40 ms, thời gian phát hiện backend DOWN khoảng 3 giây và khôi phục khoảng 4 giây. Các giá trị này chỉ đúng trong môi trường, cấu hình và mức tải đã thử.

Bài học chính của thực nghiệm là mỗi thành phần giải quyết một phần của vấn đề tính sẵn sàng. IPS cần đúng vị trí, đúng luật và có bằng chứng hành động; cân bằng tải cần backend đủ điều kiện và cơ chế kiểm tra sức khỏe. Hiệu quả phối hợp chỉ được xác định khi giữ điều kiện đầu vào nhất quán và theo dõi cả nguồn, điểm trung gian lẫn ứng dụng.

2. Hướng phát triển

Hướng phát triển gần nhất là lặp lại KB3 và KB4 thêm hai lần (R3) để đánh giá độ dao động, bổ sung nguồn truy cập hợp lệ độc lập để đánh giá mức duy trì dịch vụ và khả năng chặn nhầm, đặc biệt khi truy cập cùng URI đang được theo dõi.

Về chính sách, cần khảo sát kết hợp ngưỡng từng nguồn với quan sát tổng tải của dịch vụ, chọn giá trị dựa trên đường cơ sở thay vì ngưỡng minh họa. Về hiệu năng, có thể so sánh một backend với hai backend trên cùng cấu hình tải để định lượng lợi ích của cân bằng. Về độ tin cậy, có thể nghiên cứu dự phòng cho pfSense vì hai chức năng hiện đặt trên cùng một điểm trung gian.

Khi mở rộng sang HTTPS hoặc môi trường mạng thật, cần đánh giá lại vị trí kiểm tra, cơ chế nhìn thấy nội dung ứng dụng và giới hạn băng thông. Đây là những hướng mở rộng riêng; kết quả HTTP rõ trong mạng ảo không được xem là bằng chứng sẵn có cho các môi trường đó.

TÀI LIỆU THAM KHẢO

Các tài liệu trực tuyến dưới đây được truy cập ngày 09/11/2026. Tài liệu tham chiếu dùng Suricata 7.0.11 và HAProxy 3.2 để đối chiếu cơ chế; phần mềm thực cài của nhóm là Suricata 7.0.8 và HAProxy 2.8.3, ghi tại Bảng 3.1.

[1] CERT-EU, "DDoS Overview and Response Guide", 03/06/2024. https://cert.europa.eu/publications/security-guidance/CERT-EU_Security_Whitepaper_DDoS_17-003/

[2] Apache Software Foundation, "Apache JMeter User's Manual: Component Reference". https://jmeter.apache.org/usermanual/component_reference.html

[3] Apache Software Foundation, "Apache JMeter User's Manual: Best Practices". https://jmeter.apache.org/usermanual/best-practices.html

[4] Netgate, "The pfSense Documentation", phần yêu cầu tài nguyên của gói Snort/Suricata. https://docs.netgate.com/manuals/pfsense/en/latest/the-pfsense-documentation.pdf

[5] Open Information Security Foundation, "HTTP Keywords", Suricata 7.0.11 Documentation. https://docs.suricata.io/en/suricata-7.0.11/rules/http-keywords.html

[6] Open Information Security Foundation, "Global-Thresholds", Suricata 7.0.11 Documentation. https://docs.suricata.io/en/suricata-7.0.11/configuration/global-thresholds.html

[7] Netgate, "HAProxy package", pfSense Documentation. https://docs.netgate.com/pfsense/en/latest/packages/haproxy.html

[8] HAProxy Technologies, "HAProxy version 3.2 Configuration Manual", phần balance, frontend, backend và thống kê. https://docs.haproxy.org/3.2/configuration.html

[9] HAProxy Technologies, "Health checks", HAProxy Configuration Tutorials. https://www.haproxy.com/documentation/haproxy-configuration-tutorials/reliability/health-checks/

[10] Netgate, "Port Forwarding", pfSense Documentation. https://docs.netgate.com/pfsense/en/latest/nat/port-forwards.html

[11] Netgate và các tác giả gói pfSense Suricata, "suricata_interfaces_edit.php", kho mã nguồn FreeBSD-ports, nhánh devel; phần cấu hình Inline IPS và hỗ trợ netmap. https://github.com/pfsense/FreeBSD-ports/blob/devel/security/pfSense-pkg-suricata/files/usr/local/www/suricata/suricata_interfaces_edit.php

[12] Python Software Foundation, "http.server — HTTP servers", Python Documentation. https://docs.python.org/3/library/http.server.html

[13] Apache Software Foundation, "Apache JMeter User's Manual: Getting Started". https://jmeter.apache.org/usermanual/get-started.html

[14] Open Information Security Foundation, "Eve JSON Output", Suricata 7.0.11 Documentation. https://docs.suricata.io/en/suricata-7.0.11/output/eve/eve-json-output.html

[15] Netgate và các tác giả gói pfSense Suricata, "suricata_suppress_edit.php", kho mã nguồn FreeBSD-ports, nhánh devel. https://github.com/pfsense/FreeBSD-ports/blob/devel/security/pfSense-pkg-suricata/files/usr/local/www/suricata/suricata_suppress_edit.php

PHỤ LỤC A. HỒ SƠ CẤU HÌNH VÀ LẦN CHẠY

A.1. Thông tin cần lưu cùng báo cáo

Hồ sơ cấu hình gồm phiên bản VirtualBox, phiên bản từng hệ điều hành, Java/JMeter trên hai Kali, Python trên hai web server Ubuntu, pfSense, Suricata và HAProxy. Đối với pfSense, lưu cấu hình bằng chức năng Backup của hệ thống; nếu chia sẻ tệp ra ngoài nhóm, xử lý thông tin đăng nhập và dữ liệu không cần thiết trước khi công bố.

Tệp JMX cần được lưu riêng cho từng cấu hình tải. Nếu kế hoạch trên hai Kali giống nhau ngoại trừ SRC, ghi rõ sự khác biệt này để người đọc có thể tái lập. Ảnh cấu hình chỉ phản ánh một phần màn hình; tệp cấu hình và thông số văn bản giúp tránh thiếu những lựa chọn không hiện trong ảnh.

Bảng A.1. Phiếu ghi một lần thực nghiệm

Trường thông tin	Nội dung
Mã lần chạy	KB4_R2
Người thực hiện và thời gian	Phạm Việt Toàn; 2026-11-09, 11:31:28–11:35:41 UTC (chạy tải 11:33:58–11:35:00)
Cấu hình tải trên mỗi Kali	2 luồng × 20 vòng; ramp-up 1 giây; timer 100 ms; timeout 3000/5000 ms
Trạng thái Suricata và chính sách	WAN Inline IPS; SID 1001002; rate_filter count 20/10 giây, drop 15 giây
Trạng thái backend trước chạy	web1, web2 UP; LastChk L7OK/200; Sessions 0 trước chạy
Tệp JTL của hai nguồn	KB4_R2_K1.jtl; KB4_R2_K2.jtl
Log hai backend và IPS	backend_web1.txt; backend_web2.txt; eve_1001002.txt; alerts_1001002.txt
Thay đổi hoặc sự cố	pfSense chậm hơn Kali khoảng 4 phút; 4 timeout mỗi nguồn; không dừng sớm
Kết luận của lần chạy	6 sự kiện DROP; backend nhận 72 yêu cầu tải, giảm 10%; Hình 5.11–5.13

A.2. Quy tắc quản lý kết quả

Mỗi lần chạy dùng thư mục riêng, trong đó lưu kế hoạch, kết quả, log và ảnh. Một cấu trúc phù hợp là `KB4_R1/K1`, `KB4_R1/K2`, `KB4_R1/pfsense`, `KB4_R1/web1`, `KB4_R1/web2`. Không ghi đè kết quả của lần trước khi đang sử dụng làm đối chứng. Các ảnh gốc và ảnh đánh dấu được lưu ở hai nhóm tên phân biệt.

PHỤ LỤC B. LỆNH KIỂM TRA VÀ THU LOG

B.1. Trên Kali

```
ip -br addr
ip route
java -version
jmeter -v
curl --noproxy '*' -i --max-time 5 "http://10.10.10.1:8080/"
curl --noproxy '*' -v --max-time 5 "http://10.10.10.1:8080/ips-test.txt?case=IPS_ON_K1_R1"
```

Lệnh đầu xác nhận địa chỉ, lệnh thứ hai xem đường đi, hai lệnh tiếp theo ghi phiên bản công cụ. Hai lệnh curl dùng để kiểm tra HTTP. Thay nhãn K1 bằng K2 trên máy thứ hai; nhãn URI không thay đổi chế độ IPS. Khi cần dừng một lệnh đang chờ, dùng Ctrl+C và ghi nhận việc dừng nếu đó là một phần của lần đo.

B.2. Trên web server Ubuntu

```
ip -br addr
python3 --version
ss -ltnp | grep 8080
```

Log được đọc từ journal của dịch vụ att-web.service. Dừng và khởi động lại dịch vụ bằng systemctl; không mở hai tiến trình trên cùng cổng. Ví dụ ở Web 1 cho KB3_R2:

```
sudo mkdir -p /var/log/ddos-lab
sudo journalctl -u att-web.service --since "2026-11-09 11:29:00" --until "2026-11-09 11:31:00"
--no-pager > /var/log/ddos-lab/KB3_R2_web1.log
```

Log được dùng để tìm đúng run và URI. Thư mục log được tách khỏi thư mục web để log không trở thành tài nguyên HTTP được phục vụ. Tên tệp không phải là bằng chứng nội dung: vẫn cần kiểm tra thời gian và query trong từng dòng.

Ví dụ đếm các dòng GET của đúng mã lần chạy trong shell, với đường dẫn log điều chỉnh theo tệp đã lưu:

```
grep 'GET /load-test.txt' /var/log/ddos-lab/KB3_R2_web1.log | \
  grep -E 'run=KB3_R2(&| )' > /tmp/kb3_r2_web1_rows.txt
wc -l < /tmp/kb3_r2_web1_rows.txt
cat /tmp/kb3_r2_web1_rows.txt
```

Đây là số dòng yêu cầu được ứng dụng ghi, không mặc nhiên bằng số giao dịch duy nhất nếu có thử lại. Nếu dùng thứ tự tham số hoặc mã hóa query khác, điều chỉnh bộ lọc theo định dạng thực tế. Giữ toàn bộ log gốc để kiểm tra lại các dòng bị loại.

PHỤ LỤC C. HỒ SƠ LUẬT SURICATA

C.1. Phạm vi và trạng thái các luật

Bảng C.1. Phân biệt hai luật dùng trong báo cáo

SID	Điều kiện chính	Vai trò
1001001	Hai IP Kali; frontend:8080; URI bắt đầu /ips-test.txt	Kiểm chứng chặn trực tiếp; không dùng làm phép đo theo ngưỡng
1001002	Hai IP Kali; GET tới URI bắt đầu /load-test.txt	Theo dõi và chuyển hành động thông qua rate_filter

Nội dung đầy đủ của hai luật được trình bày tại mục 4.6.3. Khi nộp hồ sơ cấu hình, lưu bản `lab.rules` đã xuất từ hệ thống và ghi ngày áp dụng. Không giữ đồng thời một bản cũ chỉ có .10 và một bản mới cùng SID trong cùng tập luật, vì có thể tạo xung đột hoặc làm việc đối chiếu không rõ ràng.

C.2. Phân biệt trạng thái đối chứng

Trong KB3, instance inline và luật 1001002 vẫn hoạt động nhưng không áp dụng dòng rate_filter. Trong KB4, gán dòng rate_filter đã mô tả ở mục 4.6.4. Trước khi chuyển trạng thái, lưu ảnh cấu hình hiệu lực. Chờ trạng thái cũ hết ảnh hưởng, hoặc tái khởi động instance theo quy trình đã thống nhất rồi xác nhận truy cập nền trở lại bình thường.

Để tạm comment một dòng luật trong trình soạn thảo văn bản, đặt ký tự `#` ở đầu dòng. Để khôi phục, bỏ ký tự đó rồi lưu và nạp lại cấu hình. Thao tác comment chỉ có ý nghĩa nếu thay đổi được áp dụng vào instance đang chạy; việc chỉnh văn bản rồi chưa Save/Apply không thay đổi kết quả kiểm tra.

PHỤ LỤC D. CẤU HÌNH HAPROXY TƯƠNG ĐƯƠNG

Cấu hình dưới đây mô tả logic frontend và backend của báo cáo. Đây là cấu hình tham chiếu để giải thích trường trên Web GUI, không phải bản sao khẳng định đúng toàn bộ tệp pfSense đã sinh ra. Khi tái lập trên pfSense, sử dụng giao diện gói và lưu cấu hình hiệu lực; không ghi đè trực tiếp tệp do gói quản lý.

```
defaults
    mode http
    timeout connect 3s
    timeout client 10s
    timeout server 10s

frontend web_frontend
    bind 10.10.10.1:8080
    default_backend web_pool

backend web_pool
    balance roundrobin
    option httpchk GET /health.txt
    http-check expect status 200
    server web1 10.10.20.20:8080 weight 1 check inter 2s fall 3 rise 2
    server web2 10.10.20.21:8080 weight 1 check inter 2s fall 3 rise 2
```

Các giá trị timeout của HAProxy và JMeter có vai trò khác nhau. Khi JMeter hết thời gian chờ trước HAProxy, phía client có thể ghi lỗi dù proxy còn đang xử lý. Vì vậy, bảng cấu hình thực tế cần ghi cả hai phía và giữ ổn định khi đối chứng. Health check là các yêu cầu riêng của HAProxy, được loại khỏi thống kê tải do JMeter tạo.

PHỤ LỤC E. BIỂU MẪU TỔNG HỢP NHIỀU LẦN ĐO

Bảng E.1. Tổng hợp các lần chạy định lượng

Lần chạy	Tổng mẫu thực tế	Mẫu lỗi	GET tại web1 / web2	Kết quả DROP đúng SID
KB3_R1	80	0	42 / 40	0
KB3_R2	80	0	42 / 42	0
KB3_R3	—	—	—	Chưa chạy
KB4_R1	80	8	40 / 38	4 (2 / 2)
KB4_R2	80	8	37 / 37	6 (2 / 4)
KB4_R3	—	—	—	Chưa chạy

Bảng E.1 tổng hợp các lần chạy định lượng. KB3 và KB4 đều có hai lần R1, R2; R2 là lần có đủ JTL, log và ảnh nên được dùng làm kết quả chính. Các ô R3 ghi “chưa chạy” vì giới hạn thời gian của bài thực hành, không thay bằng số liệu suy đoán.

Báo cáo không vẽ thêm biểu đồ để tránh biểu diễn sai dữ liệu gốc. Bảng E.1 và các bảng 5.2–5.4 đã tách rõ tổng mẫu, số lỗi, phân bố backend và số sự kiện DROP; nếu cần biểu đồ, dùng hai nhóm cột KB3/KB4 với trục tung ghi “yêu cầu đã được ứng dụng ghi nhận”.